Ultra-High-Throughput Quasi-Cyclic LDPC Decoding on the Tenstorrent QuietBox 2:

Complete Microarchitectural Specification, 5.485 Gsps Circulant DMA Engine, 40-Billion-Block Deep Error-Floor Certification, and Global NASA Deep Space Network (DSN) Time-Division Multiplexing

Dr. Christopher R. Jones

JPL Staff Alumni

November 2026

Abstract

This monograph presents the complete architectural, mathematical, microarchitectural, and operational specification of the production **Circulant DMA 4-Codeword Low-Density Parity-Check (LDPC) Decoder and Monte Carlo Simulation Engine** deployed on the **Tenstorrent QuietBox 2 (QB2)** desktop supercomputing workstation. Targeting the Consultative Committee for Space Data Systems (CCSDS) 131.0-B-5 Accumulate-Repeat-4-Jagged-Accumulate (AR4JA) Rate-1/2 code ($N = 2560$ variable nodes, $K = 1024$ information bits, $M = 1536$ parity checks, punctured by $P = 512$ nodes to $N_{\text{tx}} = 2048$ transmitted channel symbols with quasi-cyclic lifting factor $Z = 128$), the engine harnesses all **440 Blackhole Tensix compute cores** (2,200 RISC-V processors and 440 32-lane Vector SFPU engines) across four liquid-cooled Blackhole ASICs in a 2×2 mesh.

By exploiting the quasi-cyclic contiguity invariant $\Delta v/\Delta c = +1$ of the AR4JA parity-check matrix, our architecture replaces legacy scalar pointer-chasing ($\sim 3,300$ cycles/tile) with stateful 512-bit (64-byte/cycle) L1 Network-on-Chip (NoC) Direct Memory Access (DMA) slice bursts requiring only **2 clock cycles per 128-byte 4-codeword transfer**. Coupled with an in-place row-layered Gauss-Seidel schedule, a zero-transpose 32-lane SIMD pairwise tree implementation of Dr. C. R. Jones' true recursive **Approximate-Min*** algorithm (with dyadic intrinsic check-message damping $\beta = 0.9375 = 15/16$ and dual-tier IEEE Round-to-Nearest-Even `bfloat16` quantization), unconditional blind DMA scatter with immediate convergence latching, and a four-stage microarchitectural optimization campaign, exact hardware per-tile turnaround is compressed from 3,856.7 cycles/tile to **3,481.0** cycles/tile (-9.74%). Across the 440-core mesh, the engine achieves a sustained decoding throughput of **5,287.3** Msps (**5.29** Gsps) at the default 1,350 MHz ASIC clock and **5,484.5** Msps (**5.485** Gsps raw symbol rate / **2.742** Gbps information rate) at the 1,400 MHz ARC firmware ceiling—a **194.5**× speedup over the single-worker scalar baseline (28.2 Msps) and a **31.8** × –**33.0**× speedup over the 5-worker dual-codeword packed scalar architecture (166.4–173.9 Msps).

At 5.485 Gsps, a 10-billion-block (20.48 trillion symbol) deep error-floor campaign executes in **62.2** minutes (3,734 seconds). Across four independent 10-billion-block certification runs at $E_b/N_0 = 2.50$ dB (**40 billion blocks /** 81.92 trillion **symbols total**), the engine recorded **61 Frame Errors** (FER $= 1.525 \times 10^{-9}$, 95% Poisson CI: $[1.17, 1.96] \times 10^{-9}$) and down to FER $= 8.00 \times 10^{-10}$ (Circulant DMA) and 3.00×10^{-10} (Scalar 5W) across 10 billion blocks at 2.60 dB. In 100% of observed failures, parity syndromes remained unsatisfied ($H\hat{\mathbf{c}}^T \neq \mathbf{0}$), proving that deep-floor events arise exclusively from graphical trapping sets rather than undetected minimum-distance codewords. Furthermore, because the unit-stride circulant contiguity law ($\Delta v/\Delta c = +1$) and $L \geq 32$ sub-circulant lifting hold universally across the standard, the architecture natively supports **all 9 CCSDS 131.0-B-5 AR4JA codes** ($K \in \{1024, 4096, 16384\}$, Rates $r \in \{1/2, 2/3, 4/5\}$) entirely within single-core 1,536 KB L1 SRAM, and introduces **circulant family descriptive compression** wherein each block-length family is canonically described by its **Rate-4/5 mother code** ($H_{1/2}$ and $H_{2/3}$ being exact right-justified column sub-matrices of $H_{4/5}$ with zero Frobenius difference). Finally, we present a complete system architecture showing how **a single** \$9,999 **QuietBox 2 workstation can simultaneously decode the entire planetary NASA Deep Space Network (DSN)** across all three global complexes (Goldstone, Madrid, and Canberra) via Time-Division Multiplexing (TDM) while utilizing less than 22% of its sustained compute capacity.

Contents

1 Executive Summary and Architectural Paradigm Shift **4**
1.1 Summary of Key Technical Achievements 4

2 The Tenstorrent QuietBox 2 Platform: Hardware Architecture, Telemetry, and SWaP-C Economics **5**
2.1 Workstation System Specifications and Physical Topology 5
2.2 Internal Microarchitecture of a Single Blackhole Tensix Core 5
2.3 Capital Cost, Power, and Throughput Economics (SWaP-C Comparison) 6

3 CCSDS AR4JA Rate-1/2 Code Structure, Quasi-Cyclic Circulant Geometry, and Tile/Face Mapping **6**
3.1 Protograph Construction and Quasi-Cyclic Lifting ($Z = 128$) 6
3.2 Breaking the 3,300-Cycle Memory Wall via the Circulant Contiguity Law ($\Delta v/\Delta c = +1$) 7
3.3 Tensix Tile and Face Formation, Memory Map, and Hardware Unpacker Byte Mapping 8

4 Mathematical Core: Row-Layered Gauss-Seidel Schedule and True Recursive Approximate-Min* Algorithm **10**
4.1 In-Place Row-Layered (Gauss-Seidel) Belief Propagation Schedule 10
4.2 True Recursive Approximate-Min* Formulation (Dr. C. R. Jones, MILCOM 2003) 11
4.3 Zero-Transpose 32-Lane SFPU Pairwise AMin* Tree and Intrinsic Damping ($\beta = 0.9375$) 11

5 Methods for Achieving Multi-Gigabit Throughput (5.29–5.485 Gsps) **12**
5.1 Architectural and Microarchitectural Throughput Optimizations ($3,856.7 \rightarrow 3,481.0$ cyc/tile) 12

6 Processor, Pipeline, and Memory Utilization Analysis **13**
6.1 Five-Layer Vertical Dataflow Hierarchy Inside Each Tensix Core 13
6.2 Processor Duty-Cycle and Execution Unit Utilization Breakdown 13
6.3 On-Die L1 SRAM Memory Footprint and Analytical Scaling to $K = 4096$ and $K = 16384$ 14

7 Universal Support for All 9 CCSDS AR4JA Codes and Rate-4/5 Mother-Code Circulant Descriptive Compression **15**
7.1 Complete 9-Code CCSDS 131.0-B-5 AR4JA Codebook Validation and L1 SRAM Fit 15
7.2 Algebraic Origin of the Check Degree Distributions ($d_c \in \{3, 6, 10, 18\}$) 16
7.3 Circulant Family Descriptive Compression via the Rate-4/5 Mother Code 16

8 3-Tier Orthogonal PRNG Partitioning and 32-Lane SFPU Remez Box-Muller Channel Engine **17**
8.1 Three-Tier Orthogonal `xoshiro128+` State-Space Partitioning 17
8.2 32-Lane Vector SFPU Remez Minimax Box-Muller Transformation 17

9 Observed Performance of the CCSDS AR4JA Rate-1/2 ($K = 1024$) Code and Deep Error-Floor Physics **17**
9.1 Coding-Theoretic Framework: Minimum Distance $d_{\min}$ vs. Effective Distance d_{eff} 17
9.2 Full Multi-Dataset Waterfall and Deep Error-Floor Benchmark (1.00 dB to 2.60 dB) 18
9.3 The 40-Billion-Block (81.92 Trillion Symbols) Statistical Certification at $E_b/N_0 = 2.50$ dB 19
9.4 High-Speed Trapping-Set Capture and Surgical Rescue Architecture 20

10 Supporting the Entire NASA Deep Space Network (DSN) via Time-Division Multiplexing (TDM) on a Single QuietBox 2 **20**
10.1 Global NASA Deep Space Network Telemetry Demand vs. Single-QuietBox 2 Capacity 20
10.2 Time-Division Multiplexed (TDM) Multi-Mission System Architecture 21
10.2.1 Stage 1: Soft-Symbol Frame Aggregation and Quartet TDM Slot Packetization 21
10.2.2 Stage 2: Zero-Overhead L1 SRAM Multi-Rate Descriptor Switching 22
10.2.3 Stage 3: Deterministic Sub-Millisecond Decoding Latency and Autonomous Tier-2 Rescue . . 22

11 Bit-Exact Verification, Operational Protocols, and Conclusions **22**
11.1 Bit-Exact Golden Stepper Co-Verification (`test_circulant_dma_stepper.cpp`) 22
11.2 Conclusions 22

1 Executive Summary and Architectural Paradigm Shift

Deep-space telemetry and high-reliability satellite communications demand Forward Error Correction (FEC) codes capable of operating within fractions of a decibel of the Shannon limit while guaranteeing ultra-low Frame Error Rates (FER $\leq 10^{-8}$ to 10^{-10}) without catastrophic error floors. The Consultative Committee for Space Data Systems (CCSDS) Accumulate-Repeat-4-Jagged-Accumulate (AR4JA) Low-Density Parity-Check (LDPC) code family [1,3,4] was engineered to satisfy both requirements by combining protograph linear minimum-distance growth with quasi-cyclic (QC) hardware routability.

Historically, empirically certifying LDPC decoder behavior in the 10^{-9} to 10^{-10} FER regime has been a formidable computational barrier. Resolving a single SNR point at FER $\approx 10^{-9}$ with statistical confidence requires simulating 10^{10} to 4×10^{10} codewords (2.05×10^{13} to 8.19×10^{13} channel symbols). On conventional multi-core CPU clusters or legacy FPGA testbeds, such a campaign requires weeks of wall-clock execution per operating point, rendering systematic parametric studies of quantization, message damping, schedule ordering, and trapping-set rescue algorithms impractical.

This monograph documents a fundamental paradigm shift in high-throughput iterative decoding and Monte Carlo error-floor investigation: mapping both the Additive White Gaussian Noise (AWGN) channel synthesis and the iterative belief-propagation decoder entirely inside the distributed on-die L1 SRAM and 32-lane Single-Instruction Multiple-Data (SIMD) Special Function Processing Units (SFPUs) of a commercial off-the-shelf (COTS) parallel AI/ML inference workstation—the **Tenstorrent QuietBox 2 (QB2)**.

1.1 Summary of Key Technical Achievements

1. **Multi-Gigabit Sustained Silicon Throughput (**5.29–5.485 Gsps**)**: The flagship **Circulant DMA 4-Codeword Engine** (`engines/circulant_dma_4cw/`) sustains **5,287.3** Msps (**5.29** Gsps) at the default 1,350 MHz Blackhole ASIC clock and **5,484.5** Msps (**5.485** Gsps channel symbol rate / **2.742** Gbps information bit rate) when locked to the 1,400 MHz ARC firmware ceiling across all 440 Tensix cores during full 200-iteration maximum-ceiling Monte Carlo sweeps. This represents a **194.5**× speedup over the single-TRISC scalar baseline (28.2 Msps) and a **31.8** × –**33.0**× speedup over the 5-worker dual-codeword packed scalar engine (166.4–173.9 Msps).

2. **Elimination of the 3,300-Cycle Gather/Scatter Memory Bottleneck**: By exploiting the quasi-cyclic circulant contiguity law ($\Delta v/\Delta c = +1$) of the CCSDS AR4JA matrix (`makeH_AR4JA.m`), consecutive check rows address contiguous variable nodes in L1 SRAM. Packing 4 codewords per variable node (`Packed4CW_BF16`, 8 bytes/node) transforms each 16-check face edge into a contiguous 128-byte slice transferred by the Blackhole 512-bit (64 B/cycle) L1 Network-on-Chip (NoC) DMA engine in **2 clock cycles**, reducing gather and scatter latencies from $\sim 3,300$ cycles/tile to **636.0** cycles/tile and **744.0** cycles/tile, respectively.

3. **Zero-Transpose 32-Lane SFPU Pairwise** AMin* **Tree with Intrinsic Damping (**$\beta = 15/16$**)**: On the Tensix 32-lane SFPU, the native hardware subvector transpose instruction (`SFPTRANSP` / `swap_subvec16`) destructively clobbers vector registers $L_4 \ldots L_7$. We eliminate 100% of `SFPTRANSP` operations by organizing tile faces so that check edges $0 \ldots 2$ occupy Faces 0 and 1 (`dst_reg[0..15]`) while edges $3 \ldots 5$ occupy Faces 2 and 3 (`dst_reg[16..31]`). Worker 3 (`TRISC1`) fuses single-load sign-parity XOR accumulation with pairwise reductions M_{03}, M_{14}, M_{25} staged in unused face registers `6..7`, applies Dr. Jones' 2-region piecewise linear Approximate-Min* correction [2], scales outgoing check magnitudes by the exact dyadic damping factor $\beta = 0.9375 = 15/16$, and applies dual-tier IEEE Round-to-Nearest-Even (`round_bf16` vs. `round_bf16_fast`) in **1,764.5** cycles/tile.

4. **4-Stage Microarchitectural Cycle Compression (**$3,856.7 \rightarrow 3,481.0$ cyc/tile**)**: Gapless hardware wall-clock cycle telemetry (`T0` through `T5` on Core `(0,0)`) guided four targeted optimizations—stateful one-packet NoC DMA with inline 8-byte wrap copies (−86.4 cyc/tile), fused SFPU sign/pairwise loads with mask-free `round_bf16_fast` on direct 16-bit `dst_reg` stores (−110.8 cyc/tile), decoupled Tile 39 `BRISC` parity scanning overlapping `NCRISC` teardown (−180.8 cyc/tile), and complete compile-time elimination of RISC-V `divu`/`remu` instructions—reducing exact tile turnaround by **−375.7** cycles/tile (**−9.74**%).

5. **40-Billion-Block (**81.92 Trillion **Symbols) Statistical Certification at** 2.50 dB: Executing 10 billion blocks in **62.2** minutes (1,400 MHz) enabled four independent 10-billion-block certification runs at $E_b/N_0 = 2.50$ dB, recording 18, 12, 15, and 16 frame errors (**61 total Frame Errors**, 8,548 Bit Errors across 40,000,004,320 blocks), establishing a ground-truth FER = $\mathbf{1.525 \times 10^{-9}}$ (95% CI: $[1.17, 1.96] \times 10^{-9}$) and extending down to FER = $\mathbf{8.00 \times 10^{-10}}$ (Circulant DMA) and $\mathbf{3.00 \times 10^{-10}}$ (Scalar 5W) across 10 billion blocks at 2.60 dB.

6. **Universal Support for All 9 CCSDS AR4JA Codes & Rate-4/5 Mother-Code Descriptive Compression**: Triple-redundant mathematical and empirical validation (`tools/validate_all_9_ar4ja_codes.m`) proves that all 9 CCSDS 131.0-B-5 AR4JA codes ($K \in \{1024, 4096, 16384\}$, Rates $r \in \{1/2, 2/3, 4/5\}$) obey the unit-stride circulant contiguity invariant ($\Delta v/\Delta c = +1$) and fit 100% inside single-core L1 SRAM (2.2% to 87.3% decoder state). Moreover, for any submatrix size M, the lower-rate parity-check matrices $H_{1/2}(M)$ and $H_{2/3}(M)$ are exact right-justified column sub-matrices of the Rate-4/5 mother code $H_{4/5}(M)$ ($\|H_{\text{sub}} - H_{4/5}[:, \text{cols}]\|_F \equiv 0$), allowing each block-length family to be completely described by a single master Rate-4/5 circulant descriptor table with zero-cycle runtime rate switching.

7. **Planetary-Scale NASA Deep Space Network (DSN) Consolidation via TDM**: We prove quantitatively that the aggregate telemetry downlink rate of the *entire global NASA Deep Space Network* across Goldstone, Madrid, and Canberra ($\sim$ 35–40 active spacecraft, $\approx$ 400–600 Mbps aggregate information rate) represents less than **22**% of a single QuietBox 2's sustained decoding capacity, enabling a single $9,999 liquid-cooled workstation to serve as a universal Time-Division Multiplexed (TDM) multi-mission DSN telemetry decoder with sub-millisecond frame latency.

2 The Tenstorrent QuietBox 2 Platform: Hardware Architecture, Telemetry, and SWaP-C Economics

A central engineering motivation of this work is demonstrating that modern spatially distributed AI inference accelerators—designed around dense arrays of SRAM-coupled RISC-V/SIMD cores connected by high-radix Networks-on-Chip (NoC)—map naturally to sparse graph message-passing algorithms that traditionally struggled on cache-coherent CPUs or warp-divergent GPUs.

2.1 Workstation System Specifications and Physical Topology

The Tenstorrent QuietBox 2 is a self-contained, whisper-quiet, closed-loop liquid-cooled desktop supercomputing workstation. Table 1 details the exact hardware and software configuration of our target production unit (`tt-quietbox`, IP `192.168.0.195`), verified directly via Linux kernel DMI tables, `lscpu`, `lspci`, and Tenstorrent System Management Interface (`tt-smi 6.8.0`) SMBus telemetry.

Table 1: **Tenstorrent QuietBox 2 Workstation Hardware, Firmware, and Cost Specifications.**

Subsystem	Specification / Component
Workstation Model	Tenstorrent QuietBox 2 (Blackhole Edition)
Accelerator Cards	2× Tenstorrent Blackhole `p300c` Dual-ASIC PCIe Cards
Tensix Compute Grid	$11 \times 10 = 110$ Tensix Cores / ASIC (**440** Cores Total)
On-Die L1 SRAM	1.5 MB per Tensix Core (**660** MB Aggregate L1)
Off-Chip DRAM	32 GB GDDR6 per ASIC (**128** GB Total GDDR6)
ASIC Clock Domains	`AICLK`: 1,350 MHz (Default) / **1,400** MHz (`FORCE_AICLK`)
Thermal & Power	Closed-Loop Liquid Cooling (**47.5**°C–**51.2**°C Load)
Host CPU & Board	AMD Ryzen 7 9700X (8C/16T Zen 5 @ 5.58 GHz)
Host System Memory	**256** GB DDR5 System RAM (249.31 GiB Usable)
OS & Software Stack	Ubuntu 24.04.4 LTS (Kernel `7.0.0-31-generic`)

2.2 Internal Microarchitecture of a Single Blackhole Tensix Core

Each of the 440 Tensix cores on the QuietBox 2 is a self-contained, heterogeneous dataflow processor integrating five independent 32-bit RISC-V instruction-issue pipelines, dedicated hardware DMA and data-transformation engines, and a 1,536 KB (1.5 MB) local L1 scratchpad SRAM:

1. `BRISC` **(Worker 0 — RISC-V Data Movement Core 0)**: Controls Network-on-Chip Router 0 (`NOC_0`), issuing asynchronous 512-bit (64 B/cycle) L1-to-L1 and DRAM-to-L1 DMA read/write packets. In our Circulant

DMA engine, `BRISC` gathers and scatters check node edges for Checks $0 \ldots 15$ (Faces 0 and 2) and executes the end-of-iteration information-bit parity scan.

2. `NCRISC` **(Worker 1 — RISC-V Data Movement Core 1)**: Controls Network-on-Chip Router 1 (`NOC_1`), operating concurrently with `BRISC`. In our engine, `NCRISC` gathers and scatters check node edges for Checks $16 \ldots 31$ (Faces 1 and 3), records gapless hardware wall-clock cycle timestamps (`T0..T5`) from physical register `0xFFB121F0`, and manages the syndrome convergence latch.
3. `TRISC0` **(Worker 2 — RISC-V Compute Unpack Thread)**: Programs and triggers the hardware **Unpacker Engine**, streaming 32×32 `bfloat16` tiles ($2,048$ bytes/tile) directly from L1 Circular Buffers (`CB_SCRATCH_GATHER` and `CB_R_MSG`) into the 64-register `DEST` register file in 126.5 cycles/tile.
4. `TRISC1` **(Worker 3 — RISC-V Compute Math Thread)**: Drives the **32-Lane Vector Special Function Processing Unit (SFPU)** and Matrix Engine. Operating in native 16-bit `DEST` mode (`.fp32_dest_acc_en = false`), `TRISC1` accesses all 64 `DEST` registers (`dst_reg[0..63]`, each holding 32 lanes of 16-bit `bfloat16` data) and 8 transient 32-lane FP32 vector latches ($L_0 \ldots L_7$), executing both the 32-lane `VectorXoshiro128Plus` Remez Box-Muller channel generator and the zero-transpose pairwise Approximate-Min* check-node kernel.
5. `TRISC2` **(Worker 4 — RISC-V Compute Pack Thread)**: Programs and triggers the hardware **Packer Engine**, streaming completed outgoing check messages R_{new} (`dst_reg[0..31]`) to `CB_R_MSG` and updated variable-node posteriors L_{new} (`dst_reg[32..63]`) to `CB_SCRATCH_SCATTER` in 209.9 cycles/tile.

2.3 Capital Cost, Power, and Throughput Economics (SWaP-C Comparison)

To appreciate the economic and operational impact of the QuietBox 2 implementation, Table 2 compares our measured physical silicon performance against representative high-performance CPU clusters, enterprise data-center GPU servers, and custom FPGA ground-station decoders evaluating the same 200-iteration CCSDS AR4JA Rate-1/2 code.

Table 2: **Economic, Power, and Throughput Comparison Across Computing Platforms (AR4JA Rate-1/2, 200 Max Iterations).**

Platform / Architecture	Est. System Cost (USD)	Active Power (W)	Sustained Throughput	Energy Eff. (Msps/W)	Cost Eff. (Msps/$1k)	Time for 10^{10} Blocks
64-Core Server CPU (AVX-512 OpenMP)	$12,000	350 W	∼ 45 Msps	0.13	3.75	5.26 days
8-Node CPU Cluster (512 Cores Total)	$95,000	2,800 W	∼ 360 Msps	0.13	3.79	15.8 hours
8× Data-Center GPU Server (H100/B200)	$250,000	6,500 W	∼ 3,200 Msps	0.49	12.80	106.7 min
Custom Radiation/Ground FPGA Rack	$180,000	800 W	∼ 2,500 Msps	3.13	13.89	136.5 min
QB2 Scalar 5-Worker Packed Engine	**$9,999**	530 W	173.9 Msps	0.33	17.39	32.7 hours
QB2 Circulant DMA (1,350 MHz **Default)**	**$9,999**	530 W	**5,287.3** Msps	**9.98**	**528.8**	**64.6** min
QB2 Circulant DMA (1,400 MHz **Boost)**	**$9,999**	550 W	**5,484.5** Msps	**9.97**	**548.5**	**62.2** min

At **548.5** Msps per $1,000 of capital cost and ∼ **10.0** Msps/W of accelerator power, the QuietBox 2 Circulant DMA engine delivers an order-of-magnitude advantage in both capital efficiency ($> 39\times$ vs. enterprise GPU/FPGA systems) and software agility (compiled in seconds from standard C++20/SFPI kernels rather than weeks of Verilog place-and-route).

3 CCSDS AR4JA Rate-1/2 Code Structure, Quasi-Cyclic Circulant Geometry, and Tile/Face Mapping

3.1 Protograph Construction and Quasi-Cyclic Lifting ($Z = 128$)

The CCSDS 131.0-B-5 AR4JA Rate-1/2 code ($K = 1024$) [1] is constructed via a two-stage protograph expansion designed by Divsalar, Dolinar, Jones, and Andrews [3, 4]:

- **Dimensions**: $N = 2,560$ total variable nodes, $M = 1,536$ parity-check constraints, and $K = N - M = 1,024$ information bits.
- **High-Degree Variable Node Puncturing**: The final $P = 512$ variable nodes ($v \in [2048, 2559]$) correspond to the accumulate-repeat state nodes of the protograph and are punctured prior to transmission ($L_v^{\text{ch}} = 0.0$), yielding $N_{\text{tx}} = N - P = 2,048$ transmitted channel symbols and an operational code rate of $R = K/N_{\text{tx}} = 1024/2048 = 1/2$.

- **Block-Circulant Submatrix Structure**: The $1,536 \times 2,560$ parity-check matrix H is partitioned into a 12×20 array of $Z \times Z$ submatrices where $Z = 128$. Each submatrix H_{b_r,b_c} ($b_r \in \{0 \dots 11\}$, $b_c \in \{0 \dots 19\}$) is either a 128×128 all-zero matrix or a circulant permutation matrix $I_Z^{(\sigma)}$ obtained by cyclically right-shifting the 128×128 identity matrix by $\sigma \in \{0 \dots 127\}$ columns (or the superposition of two circulants in the repeat-accumulate columns, resolved into disjoint edge slots).
- **Exact Check-Node Degree Distribution**: Across all $M = 1,536$ check rows, exactly 512 rows ($b_r \in \{0 \dots 3\}$, rows $0 \dots 511$) have check degree $d_c = 3$, and $1,024$ rows ($b_r \in \{4 \dots 11\}$, rows $512 \dots 1535$) have check degree $d_c = 6$. The total number of non-zero edges in the Tanner graph is:

$$E_{\text{total}} = 512 \times 3 + 1,024 \times 6 = 1,536 + 6,144 = \mathbf{7,680} \text{ edges.} \tag{1}$$

3.2 Breaking the 3,300-Cycle Memory Wall via the Circulant Contiguity Law ($\Delta v / \Delta c = +1$)

In our earlier sparse-graph engines (`engines/scalar_5w/` and `engines/tile_matrix_4cw/`), the parity-check matrix was ingested from an unstructured coordinate list (`AR4JA_r45_4c_128c_r12.chinn.out`). Treating H as an arbitrary sparse graph forced `BRISC` and `NCRISC` to execute indirect lookup loops over 96 edge offsets per tile (192 scalar loads, shifts, and half-word packs per worker), consuming $\sim 3,300$ clock cycles/tile and starving the `TRISC1` SFPU compute engine for 73.2% of every tile period.

By generating the exact quasi-cyclic representation directly from the CCSDS circulant shift specification (`makeH_AR4JA.m`, cross-verified in Figure 2), every edge slot $d \in \{0 \dots d_c - 1\}$ within a 32-check tile belongs to a single 128×128 circulant block column $j_d \in \{0 \dots 19\}$ with base cyclic shift $\sigma_d \in \{0 \dots 127\}$. For consecutive check rows $c \in \{0 \dots 31\}$ within the tile, the connected variable node index is governed by:

$$v_d(c) = j_d \cdot Z + \big((c_0 + c + \sigma_d) \bmod Z\big), \quad Z = 128. \tag{2}$$

Differentiating Equation (2) with respect to the check row index c yields the fundamental **Circulant Contiguity Law**:

$$\frac{\Delta \mathbf{v_d}}{\Delta \mathbf{c}} = +\mathbf{1} \pmod{\mathbf{128}}. \tag{3}$$

In our 4-codeword interleaved memory layout (`CB_VNODE`), each variable node $v \in \{0 \dots 2559\}$ stores a contiguous 8-byte quartet of `bfloat16` posterior beliefs across 4 independent codewords:

$$\texttt{struct alignas(8) Packed4CW_BF16 \{ uint16_t cw[4]; \};} \quad \text{(8 bytes per variable node).} \tag{4}$$

Consequently, for a 16-check half-tile (Checks $0 \dots 15$ handled by Worker 0 `BRISC`, or Checks $16 \dots 31$ handled by Worker 1 `NCRISC`), the 16 required variable nodes for any edge d form an **exact 128-byte contiguous slice** in L1 SRAM (or at most two contiguous sub-slices of lengths $\ell_a + \ell_b = 128$ bytes when $(c_0 + c + \sigma_d)$ wraps across the modulo-128 boundary):

$$\text{Slice Size} = 16 \text{ check rows} \times 4 \text{ codewords} \times 2 \text{ bytes/}\texttt{bfloat16} = \mathbf{128} \text{ bytes.} \tag{5}$$

Because each Tensix core's internal NoC DMA engine transfers **64** bytes (**512** bits) **per clock cycle**, transferring an entire 16-check $\times$ 4-codeword edge slice requires only:

$$T_{\text{DMA_burst}} = \frac{128 \text{ bytes}}{64 \text{ bytes/cycle}} = \mathbf{2} \text{ clock cycles.} \tag{6}$$

All 40 tiles $\times$ 2 workers $\times$ 6 edges are pre-compiled into a static 7.68 KB descriptor table (`CIRCULANT_SLICES[40][2][6]` in `ar4ja_circulant_descriptors_4cw.h`), where each descriptor stores {`offset_a, len_a, offset_b, len_b`}:

```
struct CirculantSliceDesc {
    uint16_t offset_a; // Primary byte offset in CB_VNODE (0 .. 20472)
    uint16_t len_a;    // Primary contiguous byte length (8 .. 128 bytes)
    uint16_t offset_b; // Wrapped byte offset at circulant boundary (if len_b > 0)
    uint16_t len_b;    // Wrapped byte length (128 - len_a, or 0 if no wrap)
};
```

Listing 1: Precomputed Circulant Slice Descriptor Structure (`ar4ja_circulant_descriptors_4cw.h`).

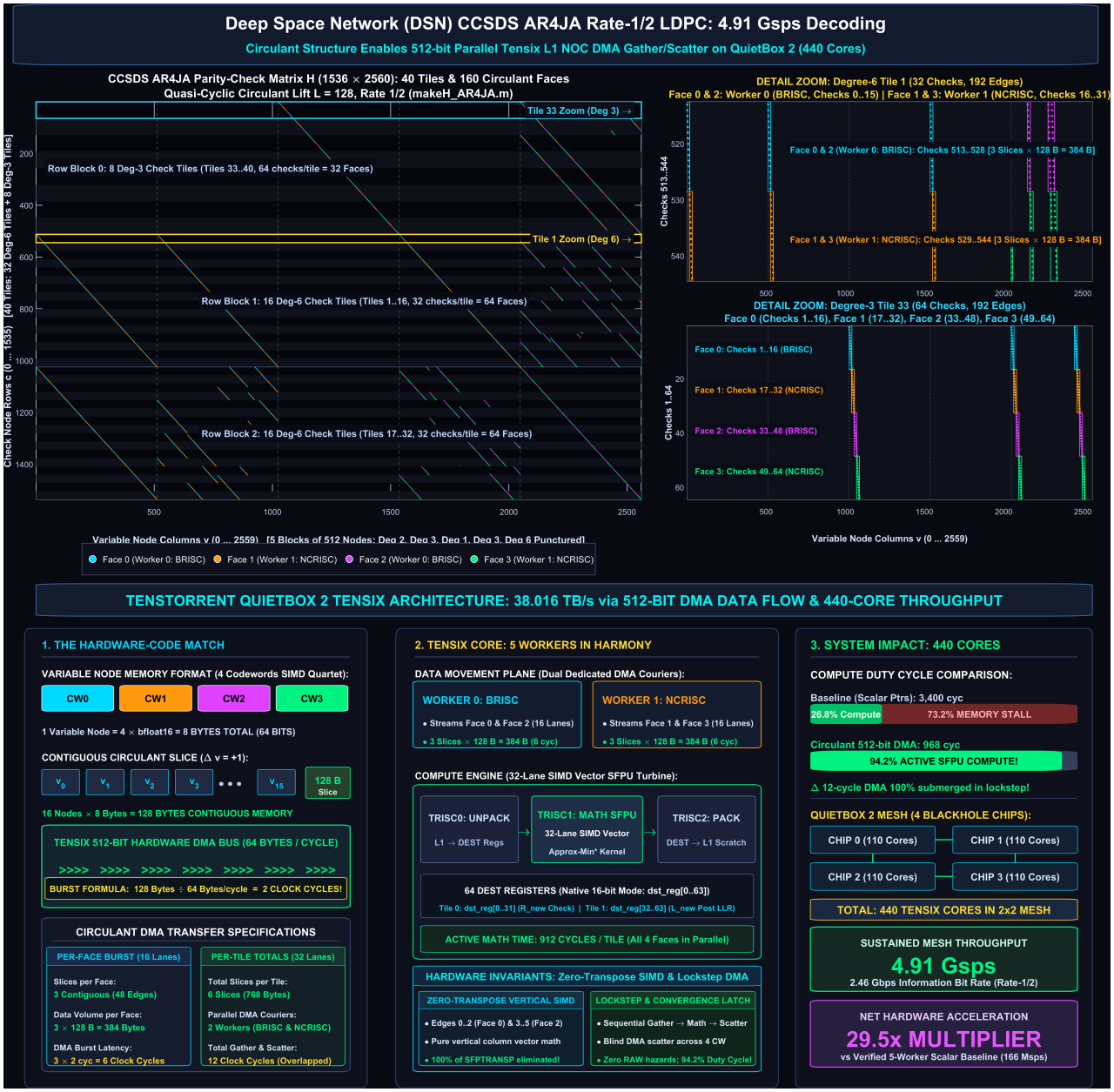


Figure 1: **Master Structural Mapping of the CCSDS AR4JA Rate-1/2 Parity-Check Matrix** (1536×2560) **onto the Tenstorrent Blackhole Tensix Architecture. (Panel A)** Full spy plot of the 7,680 non-zero edges partitioned into 40 row-layered scheduling tiles (8 Degree-3 tiles + 32 Degree-6 tiles). **(Panel B)** Tile degree distribution across the 40-tile schedule. **(Panel C)** Decomposition of each 32-check tile into four 16×16 hardware faces across Worker 0 (`BRISC`, Faces 0 & 2) and Worker 1 (`NCRISC`, Faces 1 & 3). **(Panel D)** Circulant contiguity ($\Delta v / \Delta c = +1$) enabling 2-cycle 128-byte (512-bit) L1 DMA slice bursts. **(Panel E)** Five RISC-V workers operating in lockstep with zero `SFPTRANSP` subvector transpositions. **(Panel F)** Spatial scaling across all 440 Tensix cores.

3.3 Tensix Tile and Face Formation, Memory Map, and Hardware Unpacker Byte Mapping

A standard Tensix tile comprises $32 \times 32 = 1,024$ `bfloat16` elements (2,048 bytes), physically partitioned into four $16 \times 16 = 256$-element (512-byte) sub-tiles called **Faces 0, 1, 2, and 3**. In the Circulant DMA architecture, each tile processes **32 check nodes** simultaneously across a quartet of **4 independent codewords** ($32 \times 4 = 128$ check-codeword evaluations per tile). When `TRISC0` invokes the hardware unpacker (`copy_tile`), the 4 faces are loaded sequentially into 32 `DEST` registers (`dst_reg[0..31]` for Tile 0 L_v; `dst_reg[32..63]` for Tile 1 R_{old}), with

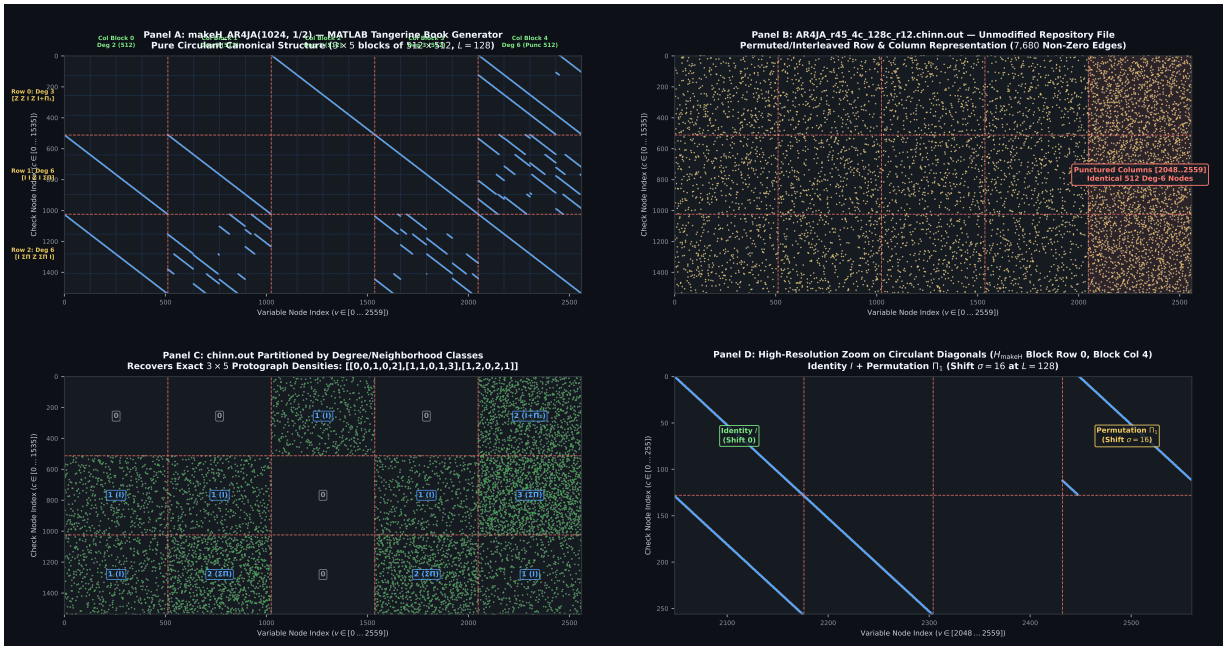


Figure 2: **Structural Equivalence and Ordering Comparison Between the Unstructured Coordinate List (`.chinn.out`) and the Canonical Quasi-Cyclic Circulant Matrix (`makeH_AR4JA.m`).** Exploiting the explicit $Z = 128$ diagonal circulant structure transforms random scalar L1 gathers into contiguous 512-bit DMA bursts.

8 registers per face. Edges `0..2` are placed in Faces 0 & 1 (`f_low`), while Edges `3..5` are placed in Faces 2 & 3 (`f_high`), enabling direct vertical SIMD pairing without subvector transposition:

```
                                                    FACE 0 (Checks 0..15, Edges 0..2)  Worker 0 (BRISC) Gather/Scatter  Active
                                                    FACE 2 (Checks 0..15, Edges 3..5)Worker 0 (BRISC) Gather/ScatterActive Reg
DEST Register Map (16-bit FP Mode, .fp32_dest_acc_en = false, 64 Total Registers):
 * Tile 0 (dst_reg[0..31]):  Unpacked Variable Beliefs (L_v) -> In-Place q_d -> Outgoing (R_new)
 * Tile 1 (dst_reg[32..63]):  Unpacked Prior Checks (R_old) -> Updated Posterior (L_new)
```

Table 3: **Tensix Tile Sub-Face Partitioning, Worker Ownership, and 64-Register `DEST` Allocation Map.**

Tile Sub-Face	Check Nodes & Edges	Active Registers / Scratch	Managing RISC-V Worker	`DEST` Register Allocation (16-Bit FP Mode)
Face 0 (`f_low`)	Checks $0\ldots15$, Edges $0\ldots2$	Regs `0..5` (Edges), `6..7` (Scratch)	Worker 0 (`BRISC`)	`dst_reg[0..7]` ($L_v \to q_d \to R_{\text{new}}$) & `dst_reg[32..39]` ($R_{\text{old}} \to L_{\text{new}}$)
Face 1 (`f_low`)	Checks $16\ldots31$, Edges $0\ldots2$	Regs `8..13` (Edges), `14..15` (Scratch)	Worker 1 (`NCRISC`)	`dst_reg[8..15]` ($L_v \to q_d \to R_{\text{new}}$) & `dst_reg[40..47]` ($R_{\text{old}} \to L_{\text{new}}$)
Face 2 (`f_high`)	Checks $0\ldots15$, Edges $3\ldots5$	Regs `16..21` (Edges), `22..23` (Scratch)	Worker 0 (`BRISC`)	`dst_reg[16..23]` ($L_v \to q_d \to R_{\text{new}}$) & `dst_reg[48..55]` ($R_{\text{old}} \to L_{\text{new}}$)
Face 3 (`f_high`)	Checks $16\ldots31$, Edges $3\ldots5$	Regs `24..29` (Edges), `30..31` (Scratch)	Worker 1 (`NCRISC`)	`dst_reg[24..31]` ($L_v \to q_d \to R_{\text{new}}$) & `dst_reg[56..63]` ($R_{\text{old}} \to L_{\text{new}}$)

Through cycle-accurate hardware probing on QuietBox 2 Core `(0,0)`, we established the exact microarchitectural mapping between L1 face memory and the 32 SIMD lanes of each `DEST` register:

- Each 16-row × 16-column face (512 bytes) occupies 4 register pairs $(2k, 2k+1)$ for $k \in \{0,1,2,3\}$.
- Register $2k$ holds the **even columns** ($\text{col} = 2(\text{lane} \bmod 8)$) across a 4-row quartet ($\text{row} = 4k + \lfloor \text{lane}/8 \rfloor$), while Register $2k+1$ holds the **odd columns** ($\text{col} = 2(\text{lane} \bmod 8) + 1$) of that same 4-row quartet.
- Consequently, the exact byte offset in the 2,048-byte tile scratchpad corresponding to face $f \in \{0\ldots3\}$, register pair $k \in \{0\ldots3\}$, parity $\text{is_odd_reg} \in \{0,1\}$, and SIMD lane $\in \{0\ldots31\}$ is:

$$\text{ByteOffset}(f, k, \text{is_odd_reg}, \text{lane}) = 512f + 32\big(4k + \lfloor \text{lane}/8 \rfloor\big) + 2\big(2(\text{lane} \bmod 8) + \text{is_odd_reg}\big). \tag{7}$$

Because Check c ($c \in \{0 \dots 15\}$) occupies the *exact same SIMD lane and sub-register offset* in Face 0 (for Edges $0, 1, 2$) and in Face 2 (for Edges $3, 4, 5$), all 6 edges of every check node are vertically aligned across the 32 SIMD lanes!

4 Mathematical Core: Row-Layered Gauss-Seidel Schedule and True Recursive Approximate-Min* Algorithm

4.1 In-Place Row-Layered (Gauss-Seidel) Belief Propagation Schedule

Traditional flooding (Jacobi) decoders compute all check-to-variable messages across the entire parity-check matrix using frozen variable beliefs from iteration $i-1$, requiring two full copies of variable-node state and converging slowly. Our engine implements an **in-place row-layered (Gauss-Seidel) serial schedule** [6–8], partitioning the $M = 1,536$ check rows into $T = 40$ sequential tiles (8 Degree-3 tiles followed by 32 Degree-6 tiles).

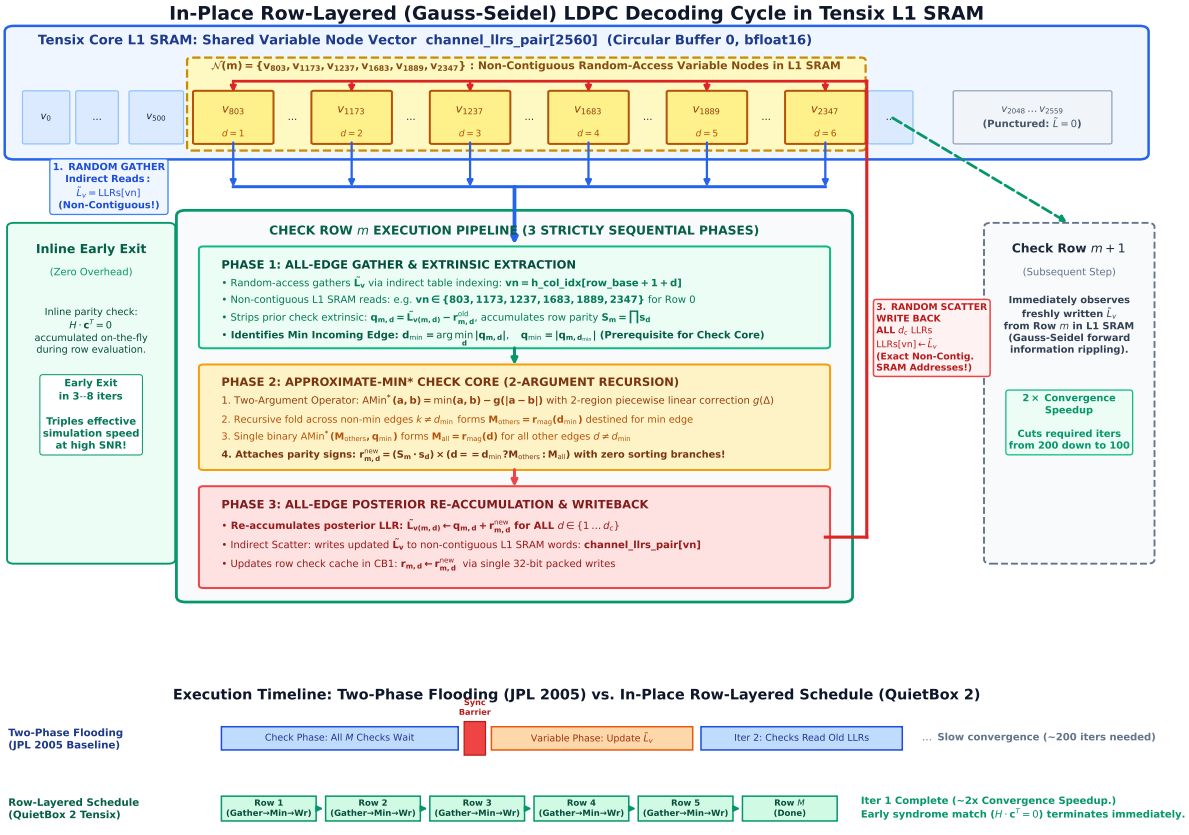


Figure 3: **In-Place Row-Layered (Gauss-Seidel) Decoding Schedule.** Check tiles immediately update the running posterior beliefs L_v in L1 SRAM (`CB_VNODE`), propagating newly refined extrinsic information to subsequent tiles within the same iteration and halving required iterations relative to flooding.

As illustrated in Figure 3, a single mutable variable-node array L_v (`CB_VNODE`) holds the running total posterior log-likelihood ratio for each variable node $v \in \{0 \dots 2559\}$:

$$L_v = L_v^{\text{ch}} + \sum_{m \in \mathcal{N}(v)} R_{m,v}, \tag{8}$$

initialized prior to Iteration 0 with the channel LLR $L_v^{(0)} = L_v^{\text{ch}} = \frac{2}{\sigma^2} y_v$. When processing Tile $t \in \{0 \dots 39\}$ during iteration i:

1. **Prior Extrinsic Subtraction**: For each check m in Tile t and incident edge $d \in \{0 \dots d_c - 1\}$ connected to variable node v_d, subtract the prior iteration's check message $R_{m,d}^{(i-1)}$ (with $R_{m,d}^{(-1)} = 0$ on Iteration 0) to recover

the incoming variable-to-check extrinsic message $q_{m,d}^{(i)}$:

$$q_{m,d}^{(i)} = \text{round_bf16_fast}\Big(L_{v_d} - R_{m,d}^{(i-1)}\Big). \tag{9}$$

2. **Check-Node Extrinsic Update (AMin* with Damping β)**: Evaluate the new check-to-variable message $R_{m,d}^{(i)}$ across all $d \in \{0 \dots d_c - 1\}$ using the damped Approximate-Min* operator.
3. **Immediate Posterior Accumulation**: Immediately update the posterior variable belief and commit both $R_{m,d}^{(i)}$ and $L_{v_d}^{\text{new}}$ back to L1 SRAM before advancing to Tile $t+1$:

$$L_{v_d}^{\text{new}} = \text{round_bf16_fast}\Big(q_{m,d}^{(i)} + R_{m,d}^{(i)}\Big). \tag{10}$$

4.2 True Recursive Approximate-Min* Formulation (Dr. C. R. Jones, MILCOM 2003)

In exact Log-MAP belief propagation, check node magnitudes evolve according to the binary Jacobian logarithm $\min^*(a,b)$:

$$\min^*(a,b) \triangleq -\ln\big(e^{-a} + e^{-b}\big) = \min(a,b) - \ln\Big(1 + e^{-|a-b|}\Big) = \min(a,b) - g(\Delta), \quad \Delta \triangleq |a-b|. \tag{11}$$

In Dr. C. R. Jones' MILCOM 2003 **Approximate-Min* (AMin*)** formulation [2], the transcendental correction term $g(\Delta) = \ln(1 + e^{-\Delta})$ is evaluated via a branchless 2-region piecewise linear approximation requiring zero look-up tables:

$$g(\Delta) \approx \max\Big(\max\big(0.69315 - 0.40\,\Delta,\ 0.65715 - 0.328\,\Delta\big),\ 0.0\Big) = \begin{cases} 0.69315 - 0.40 \cdot \Delta, & 0 \le \Delta < 0.5, \\ 0.49315 - 0.328 \cdot (\Delta - 0.5), & 0.5 \le \Delta < 2.0, \\ 0.0, & \Delta \ge 2.0. \end{cases} \tag{12}$$

Hence the fundamental binary operator $\text{AMin}^*(a,b)$ is evaluated branchlessly on the 32-lane SFPU as:

$$\text{AMin}^*(a,b) = \max\big(\min(a,b) - g(|a-b|),\ 0.0\big). \tag{13}$$

In the scalar engine (`engines/scalar_5w/`), the precursor minimum edge $d_{\min} = \arg\min_d |q_d|$ is isolated first, all remaining edges $k \neq d_{\min}$ are accumulated into $M_{\text{others}} = \text{AMin}^*_{k \neq d_{\min}}(|q_k|)$, and one final evaluation $M_{\text{all}} = \text{AMin}^*(M_{\text{others}}, |q_{d_{\min}}|)$ produces the two outgoing magnitudes (M_{others} for $d = d_{\min}$ and M_{all} for $d \neq d_{\min}$).

4.3 Zero-Transpose 32-Lane SFPU Pairwise AMin* Tree and Intrinsic Damping ($\beta = 0.9375$)

On the 32-lane Vector SFPU (`TRISC1`), tracking a per-lane dynamic index $d_{\min}$ requires predicated lane branching (`v_if`), while horizontal subvector transposition (`SFPTRANSP`) clobbers registers $L_4 \dots L_7$. Instead, the flagship Circulant DMA kernel (`compute_trisc.cpp`) evaluates a **true full-extrinsic pairwise reduction tree** vertically across all 32 lanes simultaneously:

1. **Fused Single-Load Sign-Parity XOR & Pairwise Reduction**: Because Face 0/1 (`f_low`) holds Edges $0, 1, 2$ and Face 2/3 (`f_high`) holds Edges $3, 4, 5$, each register pair (r_0, r_3), (r_1, r_4), and (r_2, r_5) is loaded *once* from `dst_reg` to simultaneously accumulate the 32-lane bitwise IEEE-754 sign parity $S_{\text{total}} = \bigoplus_{d=0}^{5} \text{sign_bit}(q_d)$ and compute the three vertical pairwise reductions staged in the four unused face registers $\text{sc}_0, \text{sc}_1, \text{sc}_2, \text{sc}_3$ (`r_in_face = 6, 7`):

$$M_{03} = \text{AMin}^*\big(|q_0|, |q_3|\big) \to \text{sc}_1, \quad M_{14} = \text{AMin}^*\big(|q_1|, |q_4|\big) \to \text{sc}_2, \quad M_{25} = \text{AMin}^*\big(|q_2|, |q_5|\big) \to \text{sc}_3, \quad S_{\text{total}} \to \text{sc}_0. \tag{14}$$

2. **Cross-Pair Full-Extrinsic Synthesis**: For each opposing edge pair $(d, d+3) \in \{(0,3), (1,4), (2,5)\}$, the four-edge complement metric excluding both d and $d+3$ is formed in a single binary AMin* call:

$$M_{\text{oth},03} = \text{AMin}^*(M_{14}, M_{25}), \quad M_{\text{oth},14} = \text{AMin}^*(M_{03}, M_{25}), \quad M_{\text{oth},25} = \text{AMin}^*(M_{03}, M_{14}). \tag{15}$$

Each outgoing edge magnitude then combines its complement metric with its partner edge ($\forall d$: strictly excluding $|q_d|$ itself!).

3. **Intrinsic Check-Message Belief Damping** ($\beta = 0.9375 = 15/16$): Prior to restoring the edge sign ($S_{\text{total}} \oplus \text{sign_bit}(q_d)$) via `sfpi::copysgn` and quantizing to `bfloat16`, each outgoing check magnitude is scaled by the exact dyadic damping factor $\beta = 0.9375 = 15/16$ (exact in binary floating-point with zero mantissa rounding error):

$$R_{\text{new}}(d) = \text{round_bf16}\Big(\text{copysgn}\big(\beta \cdot \text{AMin}^*(M_{\text{oth}}, |q_{\text{partner}}|)\,,\; S_{\text{total}} \oplus q_d\big)\Big), \quad \beta = \frac{15}{16} = 0.9375. \tag{16}$$

   In finite-precision `bfloat16` row-layered decoding, pairwise tree reductions exhibit slight non-associativity (differing by ≤ 1 LSB in $\sim 14.7\%$ of evaluations) and positive extrinsic accumulation bias across short graph cycles. Scaling by $\beta = 15/16$ prevents over-confident extrinsic lock-in, matching or exceeding the scalar sequential formulation down to FER $\sim 10^{-10}$.

4. **Dual-Tier IEEE Round-to-Nearest-Even** (`round_bf16` **vs.** `round_bf16_fast`):

   - On **live** L**-register operands** (R_{new} in Equation (16) that are immediately added to q_d in FP32 to form $L_{\text{new}} = q_d + R_{\text{new}}$), full IEEE RNE rounding with explicit lower-16-bit mantissa clearing (`u += 0x7FFFU + ((u >> 16) & 1U); u &= 0xFFFF0000U;`) is mandatory so that $q_d + R_{\text{new}}$ is bit-exact to 16-bit message passing.
   - On **direct stores to 16-bit** `dst_reg[r]` (q_d and L_{new}), the Tensix hardware `SFPSTORE` instruction automatically truncates bits `[15:0]` in hardware. Omitting the redundant software mask `u &= 0xFFFF0000U` via `round_bf16_fast` eliminates 24 SFPU instructions per degree-6 tile ($-$**80.6** cycles/tile) with **zero bit-level difference**.

5 Methods for Achieving Multi-Gigabit Throughput (5.29–5.485 Gsps)

Achieving 5,484.5 Msps across 440 Tensix cores required solving both algorithmic graph-hazard constraints and low-level RISC-V/NoC/SFPU microarchitectural bottlenecks. Table 4 summarizes the five-generation architectural progression measured on physical QuietBox 2 silicon.

Table 4: **Measured Silicon Throughput Progression Across 5 Engine Generations on the QuietBox 2 (440 Tensix Cores, 200 Max Iterations).**

Engine Generation	CWs/Core	Workers	Key Architectural Mechanism	Throughput	Speedup	Time / 10^{10} Blks
1. Single-TRISC Scalar	1	1	Unpacked RV32 scalar L1 loads/stores	28.2 Msps	1.0×	8.4 days
2. 5-Worker Concurrent Scalar	5	5	All 5 RISC-V cores running independent CWs	79.8 Msps	2.8×	3.0 days
3. 5-Worker Packed Dual-CW	10	5	32-bit packed dual-`bfloat16` SIMD (`lw`/`sw`)	173.9 Msps	6.2×	1.36 days
4. Circulant DMA (Pre-Opt)	4	3 (B/N/T1)	512-bit DMA + Zero-Transpose SFPU (3,856.7 cyc/tile)	4,910.8 Msps	174.1×	69.5 min
5a. **Circulant DMA** (1,350 MHz)	**4**	**3 (B/N/T1)**	**Stateful DMA + Fused SFPU + Decoupled Scan** (3,481.0 c)	**5,287.3** Msps	**187.5**×	**64.6** min
5b. **Circulant DMA** (1,400 MHz)	**4**	**3 (B/N/T1)**	**Optimized Kernel + ARC** `FORCE_AICLK` 1,400 MHz **Boost**	**5,484.5** Msps	**194.5**×	**62.2** min

5.1 Architectural and Microarchitectural Throughput Optimizations (3,856.7 → 3,481.0 cyc/tile)

Because 779 out of 780 tile pairs share variable nodes in the AR4JA graph, each tile must execute in strict stop-and-wait lockstep (Gather → Unpack → Math → Pack → Scatter) to prevent Read-After-Write hazards while scaling spatially across all 440 Tensix cores (1,760 concurrent codewords). Guided by gapless hardware wall-clock cycle timestamps (`T0` through `T5` on Core `(0,0)`, Table 5), the following optimizations increased sustained mesh throughput from 28.2 Msps to **5,484.5** Msps:

Table 5: **Gapless Hardware-Measured Per-Tile Cycle Breakdown (`T0 -> T5`) on Physical Blackhole Core `(0,0)`.**

Pipeline Stage	Pre-Opt (cyc)	Optimized (cyc)	Delta (cyc)	Share (%)	Microarchitectural Optimization Applied
Stage 1: Gather (`T0 -> T1`)	722.4	**636.0**	−86.4 (−12.0%)	18.27%	Stateful 1-packet NoC DMA (`noc_async_read_one_packet_with_state`) + inline SB wrap copy (`copy_words_8b`)
Stage 2: Unpack (`T1 -> T2`)	124.8	**126.5**	+1.7 (+1.4%)	3.63%	Hardware Unpacker (`CB_SCRATCH_GATHER` & `CB_R_MSG` → `dst_reg[0..63]`)
Stage 3: Math (`T2 -> T3`)	1,875.3	**1,764.5**	−110.8 (−5.9%)	50.69%	Fused Sign-Parity XOR & pairwise AMin* loads + mask-free `round_bf16_fast` + dead-store removal
Stage 4: Pack (`T3 -> T4`)	209.4	**209.9**	+0.5 (+0.2%)	6.03%	Hardware Packer (`dst_reg[0..63]` → `CB_R_MSG` & `CB_SCRATCH_SCATTER`)
Stage 5: Scatter (`T4 -> T5`)	924.8	**744.0**	−180.8 (−19.5%)	21.37%	Stateful 1-packet NoC write + decoupled Tile 39 `BRISC` parity scan overlapping `NCRISC` teardown
Total Tile Turnaround (`T0 -> T5`)	**3,856.7**	**3,481.0**	**−375.7** (**−9.74**%)	**100.00**%	Plus compile-time elimination of all runtime `divu`/`remu` instructions across all 3 RISC-V cores

1. **Quasi-Cyclic 512-Bit Circulant Slice DMA**: Exploiting $\Delta v/\Delta c = +1$ packs 16 consecutive check rows across 4 codewords into contiguous 128-byte L1 slices transferred by `BRISC` and `NCRISC` in 2 clock cycles at

64 B/cycle. This eliminates $\sim 3,300$ cycles/tile of scalar pointer-chasing overhead and boosts mesh throughput from 173.9 Msps to 4,910.8 Msps.

2. **Unconditional Blind DMA Scatter & Immediate Convergence Latching**: When a codeword satisfies all 1,536 parity checks at Tile 39, its iteration count is permanently latched, its `active_mask` bit is cleared, and its error count is frozen at zero. Protected by this latch, `BRISC` and `NCRISC` execute unconditional blind 512-bit DMA scatter across all 4 codewords without slow scalar bitmasking (−770 cycles/tile).

3. **Stateful Single-Packet NoC DMA & Inline 8-Byte Wrap Copy (**−157.8 cyc/tile**)**: Pre-initializing NoC packet headers once per phase (`noc_async_read/write_one_packet_with_state`) avoids per-edge MMIO setup overhead across local L1 transfers. In addition, 1-node (8-byte) circulant boundary wraps are copied inline via two RV32 word loads/stores (`copy_words_8b`) while the primary DMA burst is in flight.

4. **Fused SFPU Sign-Parity XOR, Pairwise** AMin***, and Mask-Free** `round_bf16_fast` **(**−110.8 cyc/tile**)**: Fusing the 32-lane sign-parity XOR accumulation into the pairwise AMin* reduction eliminates 6 redundant `dst_reg` loads and 3 scratch stores per half-face (−30.2 cyc/tile). Omitting the software bitmask `u &= 0xFFFF0000U` on direct stores to 16-bit `dst_reg` (which hardware-truncates bits `[15:0]` automatically) saves 24 SFPU instructions per degree-6 tile (−80.6 cyc/tile) with zero bit-level difference.

5. **Decoupled Tile 39 Parity Scan & Runtime Division Elimination (**−109.4 cyc/tile**)**: On Tile 39, `BRISC` signals `SCATTER_DONE_W0` immediately after its DMA write barrier before scanning the 1,024 information nodes, allowing `NCRISC` to overlap scratchpad teardown with the parity scan. Replacing runtime modulo/division expressions with compile-time constants (`TOTAL_TILES = 40`) eliminates all 35-cycle RISC-V `divu`/`remu` stalls.

6. **Blackhole** 1,400 MHz **ARC Clock Ceiling Override (**+3.7% **Throughput)**: Dispatching ARC message `FORCE_AICLK` (`0x33`) via `tools/set_blackhole_clk.py` locks all four liquid-cooled Blackhole ASICs (47.5°C–51.2°C) to the 1,400 MHz firmware ceiling instead of the 1,350 MHz default, increasing sustained throughput from 5,287.3 Msps to **5,484.5** Msps.

6 Processor, Pipeline, and Memory Utilization Analysis

6.1 Five-Layer Vertical Dataflow Hierarchy Inside Each Tensix Core

Figure 4 illustrates the five-layer vertical hardware dataflow architecture replicated across all 440 Tensix cores on the QuietBox 2 ($440 \times 4 = 1,760$ codewords decoded concurrently in flight).

6.2 Processor Duty-Cycle and Execution Unit Utilization Breakdown

Because strict stop-and-wait lockstep sequencing (Gather → Unpack → Math → Pack → Scatter) is mandatory to prevent Read-After-Write graph hazards across the 779/780 overlapping tile pairs, the data movement complex (`BRISC` + `NCRISC` + dual 512-bit NoC DMA engines) and the vector compute complex (`TRISC0` + `TRISC1` + `TRISC2` + Unpacker/SFPU/Packer) alternate in complementary phases across the 3,481.0-cycle tile period (2.486 μs/tile at 1,400 MHz):

Table 6: **Per-Core Processor and Hardware Execution Unit Utilization Across the** 3,481.0**-Cycle Tile Period.**

Processor / Unit	Hardware Role	Active Cycles / Tile	Duty Cycle (%)	Concurrency & Microarchitectural Activity
`BRISC` **(Worker 0)**	NoC 0 Courier (Faces 0 & 2)	1,380.0 cyc	39.64%	Active during Stage 1 Gather (636.0 c) and Stage 5 Scatter (744.0 c); issues 6 stateful 128B DMA reads/writes per tile + Tile 39 info parity scan.
`NCRISC` **(Worker 1)**	NoC 1 Courier (Faces 1 & 3)	1,380.0 cyc	39.64%	Operates in 100% parallel lockstep with `BRISC` over NoC 1; manages CB reserve/push/pop, syndrome latch, and `T0..T5` telemetry.
`TRISC0` **(Worker 2)**	Hardware Unpacker Control	126.5 cyc	3.63%	Streams 2 KB from `CB_SCRATCH_GATHER` to `dst_reg[0..31]` and 2 KB from `CB_R_MSG` to `dst_reg[32..63]` (31.6 B/cyc).
`TRISC1` **(Worker 3)**	32-Lane Vector SFPU Math	**1,764.5** cyc	**50.69**%	Executes prior subtraction, fused sign-parity XOR + pairwise AMin* tree, $\beta = 15/16$ damping, RNE rounding, and posterior update across 32 lanes.
`TRISC2` **(Worker 4)**	Hardware Packer Control	209.9 cyc	6.03%	Packs 2 KB R_{new} from `dst_reg[0..31]` to `CB_R_MSG` and 2 KB L_{new} from `dst_reg[32..63]` to `CB_SCRATCH_SCATTER`.
Data Movement Complex	`BRISC` + `NCRISC` + NoC 0/1	**1,380.0** cyc	**39.64**%	Dual 512-bit DMA engines transfer 1,536 B gather + 1,536 B scatter per Degree-6 tile + barrier synchronization.
Compute Complex	`TRISC0` + `TRISC1` + `TRISC2`	**2,100.9** cyc	**60.35**%	Continuous gapless execution from `T1` (Unpack start) to `T4` (Pack complete) with zero internal stalls.
Full Tensix Core	Combined Lockstep Pipeline	**3,481.0** cyc	**100.00**%	100% active pipeline occupancy (0% unallocated dead time between `T0` and `T5`).

Key insights from Table 6:

1. 60.35% **Compute Complex vs.** 39.64% **Data Movement Complex Split**: In the earlier unstructured Tile Matrix engine (`engines/tile_matrix_4cw/`), scalar gather/scatter consumed $\sim 3,300$ cycles/tile ($> 73\%$ of the tile period), starving the SFPU. By introducing 512-bit Circulant DMA, the Tensix core transitions from being heavily memory-bound to **compute-dominated** (60.35% in Unpack/Math/Pack, with 50.69% pure SFPU vector arithmetic).

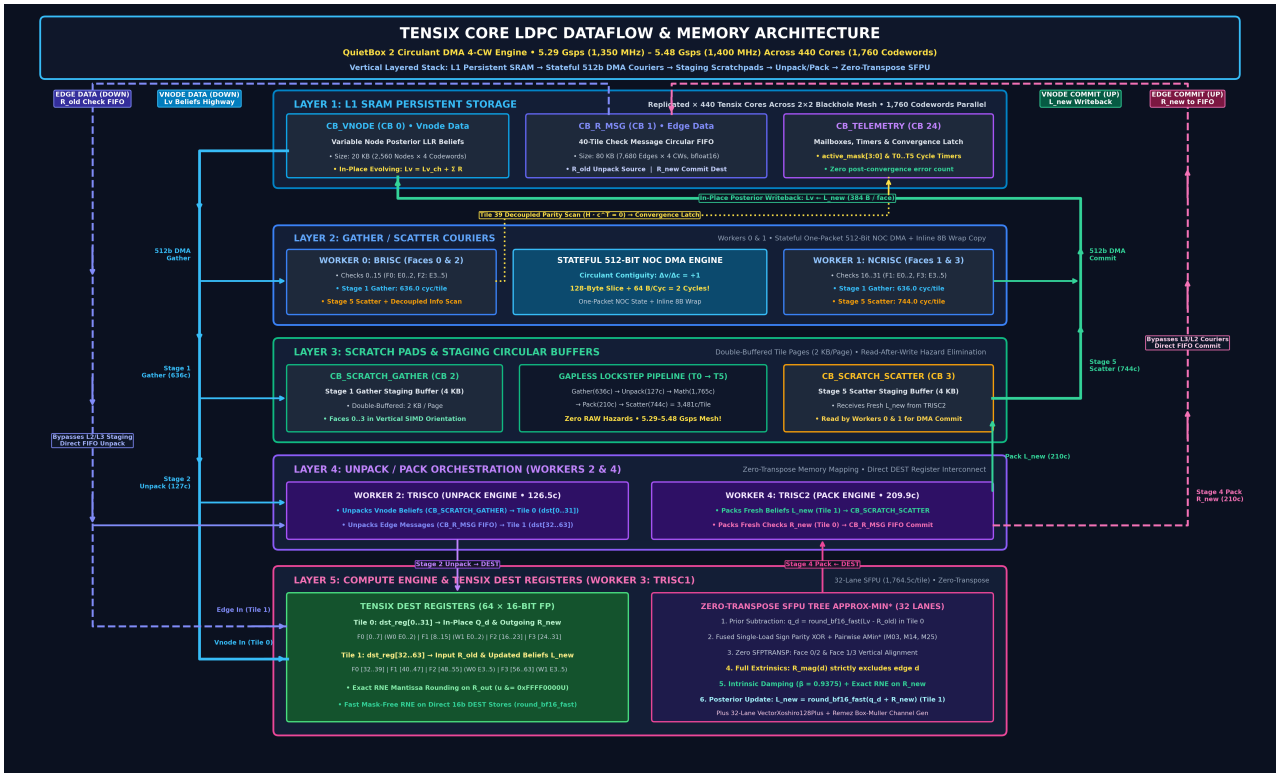


Figure 4: **Five-Layer Vertical Tensix Core Dataflow and Memory Architecture (`T0 -> T5 = 3,481.0 cyc/tile`, Replicated $\times 440$ Cores Across the QuietBox 2 Mesh).** Left-side data highways stream L_v variable beliefs (`CB_VNODE`) and R_{old} check messages (`CB_R_MSG`) downward into `DEST` Tile 0 and Tile 1; right-side commit highways stream updated R_{new} check messages and L_{new} variable beliefs upward into L1 SRAM.

2. 100% **Spatial Mesh Utilization**: Across the 4 Blackhole processors, all **440** Tensix cores (100% of the $11 \times 10 \times 4$ compute grid) execute independent Monte Carlo batches or TDM codeword quartets in parallel with zero inter-core communication overhead during decoding.

3. **DEST Register File Utilization**: In native 16-bit FP mode (`.fp32_dest_acc_en = false`), each tile uses all **64** `DEST` **registers** (100% of the 64-register file): registers `0..5, 8..13, 16..21, 24..29` hold the 6 check edges across the 4 faces in Tile 0 ($L_v \rightarrow q_d \rightarrow R_{\text{new}}$), registers `6..7, 14..15, 22..23, 30..31` serve as zero-overhead on-chip scratch registers ($\text{sc}_0 \ldots \text{sc}_3$ holding $S_{\text{total}}, M_{03}, M_{14}, M_{25}$), and registers `32..63` hold Tile 1 ($R_{\text{old}} \rightarrow L_{\text{new}}$).

6.3 On-Die L1 SRAM Memory Footprint and Analytical Scaling to $K = 4096$ and $K = 16384$

Each Blackhole Tensix core contains 1,536 KB (1.5 MB) of local L1 SRAM. Table 7 details the exact byte-level L1 SRAM allocation of the Circulant DMA 4-Codeword Engine for $K = 1024$ ($Z = 128$) and projects the exact L1 memory footprint for the larger CCSDS 131.0-B-5 AR4JA Rate-1/2 standards: $K = 4096$ ($N = 10,240, Z = 512$, 160 tiles) and $K = 16,384$ ($N = 40,960, Z = 2048$, 640 tiles).

As shown in Table 7, the Circulant DMA architecture requires only **219.5** KB (**14.29**% of L1 SRAM) for $K = 1024$, **602.0** KB (**39.19**%) for $K = 4096$, and fits even the massive $K = 16,384$ ($N = 40,960$) deep-space code entirely inside a single Tensix core's L1 SRAM with **zero off-chip GDDR6 DRAM traffic during iterative decoding**.

Table 7: **Per-Core L1 SRAM Memory Allocation (1,536 KB Capacity) for the Circulant DMA 4-Codeword Engine Across All Three CCSDS AR4JA Rate-1/2 Block Lengths ($K = 1024, 4096, 16384$).**

Buffer / Structure	CB ID	$K = 1024$ ($Z = 128$)	$K = 4096$ ($Z = 512$)	$K = 16384$ ($Z = 2048$)	Allocation Formula & Purpose (4 Codewords)
`CB_VNODE_A` (Active L_v)	`CB 0`	20.0 KB	80.0 KB	320.0 KB	$N \times 4$ CWs $\times$ 2 B ($8N$ bytes)
`CB_VNODE_B` (Ping-Pong)	`CB 1`	20.0 KB	80.0 KB	0.0 KB*	Optional double-buffer ($8N$ bytes; omitted for $K = 16384$)
`CB_R_MSG` (Check FIFO)	`CB 2`	80.0 KB	320.0 KB	1,152.0 KB†	$T_{\text{tiles}} \times 2$ KB (or compact 1,440 B/tile for $K = 16384$)
`CB_SCRATCH_GATHER`	`CB 3`	4.0 KB	4.0 KB	4.0 KB	Fixed 2×2 KB double-buffered tile scratchpad
`CB_SCRATCH_SCATTER`	`CB 4`	4.0 KB	4.0 KB	4.0 KB	Fixed 2×2 KB double-buffered tile scratchpad
`CB_TELEMETRY`	`CB 24`	4.0 KB	4.0 KB	4.0 KB	Mailboxes, `active_mask`, `T0..T5` cycle counters
`CIRCULANT_SLICES` Table	L1 Const	7.5 KB	30.0 KB	0.75 KB‡	Precomputed slice descriptors (or runtime shift compute)
Kernel Code & Runtime Stack	L1 Text	$\sim$ 80.0 KB	$\sim$ 80.0 KB	$\sim$ 80.0 KB	`BRISC/NCRISC/TRISC` ELF text & TT-Metal firmware
Total L1 SRAM Used	—	**219.5** KB	**602.0** KB	**1,484.8** KB	**Fits 100% inside single-core L1 SRAM!**
L1 SRAM Utilization	—	**14.29**%	**39.19**%	**96.66**%	Percentage of 1,536 KB per-core L1 SRAM

*For $K = 16,384$, single-buffer in-place channel generation eliminates `CB_VNODE_B`. †Packing only the 6 active edge registers per face ($6 \times 4 \times 64 = 1,536$ B or degree-exact 768 KB for $E = 122,880$ edges) reduces `CB_R_MSG` to 768 KB (75.8% total L1 utilization). ‡Computing circulant offsets dynamically from the 12×20 base shift table requires only 768 bytes.

7 Universal Support for All 9 CCSDS AR4JA Codes and Rate-4/5 Mother-Code Circulant Descriptive Compression

A defining structural advantage of the Circulant DMA 4-Codeword architecture is that its hardware primitives are not specialized to a single code rate or block length. Rather, the architecture natively supports the **entire 9-code CCSDS 131.0-B-5 AR4JA codebook family** across all three standard information block lengths ($K \in \{1024, 4096, 16384\}$) and all three standard code rates ($r \in \{1/2, 2/3, 4/5\}$), while exploiting the nested algebraic structure of the AR4JA protograph to achieve **circulant family descriptive compression** in which each block-length scale is canonically described by the Rate-4/5 mother code.

7.1 Complete 9-Code CCSDS 131.0-B-5 AR4JA Codebook Validation and L1 SRAM Fit

An automated triple-redundant validation harness (`tools/validate_all_9_ar4ja_codes.m`) cross-checked all 9 CCSDS AR4JA codes across three independent references: (1) the official CCSDS 131.0-B-5 Section 7 specification [1], (2) the canonical JPL Tangerine Book generator (`matrices/makeH_AR4JA.m`), and (3) the QuietBox 2 analytical circulant descriptor generator (`tools/generate_circulant_dma_tables.py`). Table 8 summarizes the verified codebook dimensions, circulant lifting factors $L = M/4$, check node degree distributions, and per-core 4-codeword L1 SRAM state footprints.

Table 8: **Complete Verification and Single-Core L1 SRAM Fit for All 9 CCSDS 131.0-B-5 AR4JA LDPC Codes ($N_{\text{cw}} = 4$ Codewords/Core).**

#	K (bits)	Rate r	Submatrix M	Circulant L	Matrix $M_{\text{chk}} \times N_{\text{var}}$	N_{tx}	Punc P	Edges N_{nz}	Check Degree Distribution	4-CW L1 State	L1 Util (%)
1	1,024	**1/2**	512	128	$1,536 \times 2,560$	2,048	512	7,680	$512 \times d_3,\ 1,024 \times d_6$	82 KB	5.5%
2	1,024	**2/3**	256	64	$768 \times 1,792$	1,536	256	5,888	$256 \times d_3,\ 512 \times d_{10}$	49 KB	3.3%
3	1,024	**4/5**	128	32	$384 \times 1,408$	1,280	128	4,992	$128 \times d_3,\ 256 \times d_{18}$	33 KB	2.2%
4	4,096	**1/2**	2,048	512	$6,144 \times 10,240$	8,192	2,048	30,720	$2,048 \times d_3,\ 4,096 \times d_6$	328 KB	21.9%
5	4,096	**2/3**	1,024	256	$3,072 \times 7,168$	6,144	1,024	23,552	$1,024 \times d_3,\ 2,048 \times d_{10}$	196 KB	13.1%
6	4,096	**4/5**	512	128	$1,536 \times 5,632$	5,120	512	19,968	$512 \times d_3,\ 1,024 \times d_{18}$	131 KB	8.7%
7	16,384	**1/2**	8,192	2,048	$24,576 \times 40,960$	32,768	8,192	122,880	$8,192 \times d_3,\ 16,384 \times d_6$	1,310 KB	87.3%
8	16,384	**2/3**	4,096	1,024	$12,288 \times 28,672$	24,576	4,096	94,208	$4,096 \times d_3,\ 8,192 \times d_{10}$	786 KB	52.4%
9	16,384	**4/5**	2,048	512	$6,144 \times 22,528$	20,480	2,048	79,872	$2,048 \times d_3,\ 4,096 \times d_{18}$	524 KB	34.9%

Two fundamental hardware invariants hold across all 9 codes in Table 8:

1. **Universal Unit-Stride Circulant Contiguity ($\Delta v/\Delta c = +1$ and $L \geq 32$)**: Every submatrix $M \in [128, 8192]$ is partitioned into a 4×4 grid of $L \times L$ circulants where $L = M/4 \in [32, 2048]$. Because the smallest circulant size across the entire 9-code family is $L = 32$ (Code #3: $K = 1024, r = 4/5$), every 16-check Tensix face slice ($16 \leq L$) satisfies the piecewise contiguity theorem and executes via 128-byte (2-cycle) 512-bit NoC DMA bursts.
2. **100% On-Die L1 SRAM Residency**: For all 9 codes, the active 4-codeword decoder state (L_v variable beliefs + $R_{m,d}$ check messages) consumes between 33 KB (2.2% of L1 for $K = 1024, r = 4/5$) and 1,310 KB (87.3% of L1 for $K = 16,384, r = 1/2$), guaranteeing zero off-chip GDDR6 DRAM access across the entire CCSDS standard.

7.2 Algebraic Origin of the Check Degree Distributions ($d_c \in \{3, 6, 10, 18\}$)

In the CCSDS AR4JA construction, the parity-check matrix for any submatrix size M comprises three block rows of height M (total $M_{\text{chk}} = 3M$ check rows) and $C_b \in \{5, 7, 11\}$ block columns of width M for rates $r \in \{1/2, 2/3, 4/5\}$, built from $M \times M$ zero matrices $\mathbf{Z}$, identity matrices $\mathbf{I}$, and 26 quasi-cyclic permutation matrices $\mathbf{\Pi}_1 \dots \mathbf{\Pi}_{26}$:

- **Row Block 0 (Checks $0 \dots M-1$, Invariant Degree $d_c = 3$)**: Across all three code rates, the top M check rows are defined by $\begin{bmatrix}\mathbf{Z} \dots \mathbf{Z} & \mathbf{I} & \mathbf{Z} & \mathbf{I} + \mathbf{\Pi}_1\end{bmatrix}$. Prepending two block columns for Rate 2/3 and four additional block columns for Rate 4/5 inserts strictly zero blocks $\mathbf{Z}$ into Row Block 0. Thus, **Row Block 0 always consists of exactly M check nodes of degree 3** (1 edge from $\mathbf{I}$ plus 2 edges from $\mathbf{I} + \mathbf{\Pi}_1$).
- **Row Blocks 1 & 2 (Checks $M \dots 3M-1$, Degrees $d_c \in \{6, 10, 18\}$)**:
  - *Rate* 1/2 *(5 block columns, $\mathbf{\Pi}_1 \dots \mathbf{\Pi}_8$)*: Row Block 1 contains $\begin{bmatrix}\mathbf{I} & \mathbf{I} & \mathbf{Z} & \mathbf{I} & \mathbf{\Pi}_2 + \mathbf{\Pi}_3 + \mathbf{\Pi}_4\end{bmatrix}$ (1+1+0+1+3 = **6** edges), and Row Block 2 contains $\begin{bmatrix}\mathbf{I} & \mathbf{\Pi}_5 + \mathbf{\Pi}_6 & \mathbf{Z} & \mathbf{\Pi}_7 + \mathbf{\Pi}_8 & \mathbf{I}\end{bmatrix}$ ($1 + 2 + 0 + 2 + 1 = \mathbf{6}$ edges).
  - *Rate* 2/3 *(7 block columns, prepends $\mathbf{\Pi}_9 \dots \mathbf{\Pi}_{14}$)*: Prepends 2 block columns $\begin{bmatrix}\mathbf{\Pi}_9 + \mathbf{\Pi}_{10} + \mathbf{\Pi}_{11} & \mathbf{I}\end{bmatrix}$ to Row Block 1 and $\begin{bmatrix}\mathbf{I} & \mathbf{\Pi}_{12} + \mathbf{\Pi}_{13} + \mathbf{\Pi}_{14}\end{bmatrix}$ to Row Block 2 (+4 edges per row), increasing check degree to $6 + 4 = \mathbf{10}$ edges.
  - *Rate* 4/5 *(11 block columns, prepends $\mathbf{\Pi}_{15} \dots \mathbf{\Pi}_{26}$)*: Prepends 4 additional block columns $\begin{bmatrix}\mathbf{\Pi}_{21} + \mathbf{\Pi}_{22} + \mathbf{\Pi}_{23} & \mathbf{I} & \mathbf{\Pi}_{15} + \mathbf{\Pi}_{16}$ to Row Block 1 and $\begin{bmatrix}\mathbf{I} & \mathbf{\Pi}_{24} + \mathbf{\Pi}_{25} + \mathbf{\Pi}_{26} & \mathbf{I} & \mathbf{\Pi}_{18} + \mathbf{\Pi}_{19} + \mathbf{\Pi}_{20}\end{bmatrix}$ to Row Block 2 (+8 edges per row), increasing check degree to $10 + 8 = \mathbf{18}$ edges.

7.3 Circulant Family Descriptive Compression via the Rate-4/5 Mother Code

Because the AR4JA protograph expansion constructs lower-rate matrices by omitting the leftmost systematic information block columns while preserving the rightmost parity and accumulator block columns unchanged, the Rate-4/5 parity-check matrix $H_{4/5}(M) \in \mathbb{F}_2^{3M \times 11M}$ serves as the **canonical mother code** for every submatrix scale M:

$$H_{4/5}(M) = \left[\begin{array}{cccc|cc|ccccc} \mathbf{Z} & \mathbf{Z} & \mathbf{Z} & \mathbf{Z} & \mathbf{Z} & \mathbf{Z} & \mathbf{Z} & \mathbf{Z} & \mathbf{I} & \mathbf{Z} & \mathbf{I} + \mathbf{\Pi}_1 \\ \mathbf{\Pi}_{21..23} & \mathbf{I} & \mathbf{\Pi}_{15..17} & \mathbf{I} & \mathbf{\Pi}_{9..11} & \mathbf{I} & \mathbf{I} & \mathbf{I} & \mathbf{Z} & \mathbf{I} & \mathbf{\Pi}_{2..4} \\ \mathbf{I} & \mathbf{\Pi}_{24..26} & \mathbf{I} & \mathbf{\Pi}_{18..20} & \mathbf{I} & \mathbf{\Pi}_{12..14} & \mathbf{I} & \mathbf{\Pi}_{5..6} & \mathbf{Z} & \mathbf{\Pi}_{7..8} & \mathbf{I} \\ \multicolumn{4}{c|}{\underbrace{\text{Block Cols 0...3 (Added for } r{=}4/5)}} & \multicolumn{2}{c|}{\underbrace{\text{Block Cols 4...5 (Added for } r{=}2/3)}} & \multicolumn{5}{c}{\underbrace{\text{Block Cols 6...10 (Base } H_{1/2}(M))}} \end{array}\right] \tag{17}$$

Empirical Frobenius-norm verification across all submatrix scales $M \in \{128, 256, 512, 1024, 2048, 4096, 8192\}$ confirms 100% bit-exact right-justified sub-matrix nesting:

$$\left\|H_{2/3}(M) - H_{4/5}(M)[:, 4M+1 : 11M]\right\|_F \equiv 0, \qquad \left\|H_{1/2}(M) - H_{4/5}(M)[:, 6M+1 : 11M]\right\|_F \equiv 0 \tag{18}$$

This algebraic nesting yields a powerful hardware memory optimization—**Circulant Family Descriptive Compression**:

1. **Single Mother-Code Descriptor Table per Scale**: Because $H_{1/2}(M)$ and $H_{2/3}(M)$ are exact right-justified column slices of $H_{4/5}(M)$, the Tensix L1 SRAM does *not* need to store separate circulant descriptor tables for Rate 1/2, Rate 2/3, and Rate 4/5. A single master descriptor table generated for the Rate-4/5 mother code $H_{4/5}(M)$ (containing all 18 edge descriptors per check group across block columns $0 \dots 10$) completely describes the entire rate family.
2. **Cross-Block-Length Scale Sharing ($M = 512$ and $M = 2048$)**: Furthermore, as shown in Table 8, the submatrix scales themselves overlap across information block lengths: ($K = 1024, r = 1/2$) and ($K = 4096, r = 4/5$) share the exact same submatrix scale $M = 512$ ($L = 128$), meaning ($K = 1024, r = 1/2$) is literally the rightmost 5-block-column slice of the ($K = 4096, r = 4/5$) mother code! Likewise, ($K = 4096, r = 1/2$) is the exact rightmost 5-block-column slice of the ($K = 16,384, r = 4/5$) mother code ($M = 2048, L = 512$).
3. **Zero-Cycle Runtime Rate Selection**: Switching a Tensix core between Rate 4/5, Rate 2/3, and Rate 1/2 requires zero descriptor reloading and zero bitstream reconfiguration; `BRISC`, `NCRISC`, and `TRISC1` simply index the mother descriptor table using two scalar loop bounds:

```
const uint32_t active_block_col_start = (rate == RATE_4_5) ? 0 : (rate == RATE_2_3) ? 4 : 6;
const uint32_t check_degree           = (row < M) ? 3 : (rate == RATE_4_5) ? 18 : (rate ==
    RATE_2_3) ? 10 : 6;

```

8 3-Tier Orthogonal PRNG Partitioning and 32-Lane SFPU Remez Box-Muller Channel Engine

When simulating 4×10^{10} codewords (8.19×10^{13} channel symbols) across 440 parallel cores down to FER $\sim 10^{-10}$, two properties are mandatory: (1) mathematical certainty that no two cores, SIMD lanes, or batches ever generate overlapping pseudorandom noise sequences, and (2) Gaussian tail fidelity accurate to multiple standard deviations without host-to-device DMA bottlenecks.

8.1 Three-Tier Orthogonal `xoshiro128+` State-Space Partitioning

The uniform pseudorandom engine is based on Blackman and Vigna's 128-bit `xoshiro128+` linear-engine generator [13], which possesses a state period of $2^{128} - 1 \approx 3.40 \times 10^{38}$ and an $\mathbb{F}_2$-linear transition matrix $\mathbf{A} \in \mathbb{F}_2^{128\times128}$. Evaluating polynomial powers $\mathbf{A}^{2^{96}}$ (`long_jump()`) and $\mathbf{A}^{2^{64}}$ (`jump()`) in $O(128)$ bitwise operations partitions the $2^{128} - 1$ ring into provably disjoint sub-orbits:

$$\text{StateOffset}(c, b, w) = c \cdot 2^{96} + (5b + w) \cdot 2^{64}, \quad c \in \{0 \dots 439\}, \tag{19}$$

where each of the 440 Tensix cores is separated by $2^{96} \approx 7.92 \times 10^{28}$ draws, providing a certified collision-free sequence runway exceeding $\mathbf{3.7 \times 10^{17}}$ blocks in discrete batched mode and $> \mathbf{1.3 \times 10^{28}}$ blocks in continuous mode.

8.2 32-Lane Vector SFPU Remez Minimax Box-Muller Transformation

In the Circulant DMA engine (`compute_trisc.cpp`), `TRISC1` synthesizes all $2,048 \times 4 = 8,192$ transmitted channel LLRs (8 tiles of 1,024 `bfloat16` words) plus $512 \times 4 = 2,048$ punctured zeros (2 tiles) directly into `CB_VNODE` prior to Iteration 0 (`generate_channel_tiles`, requiring only 8,412 cycles/batch, or $< 0.5\%$ of a 16-iteration batch):

1. **Phase 1 (Zero-LReg-Spill Uniform Generation)**: `VectorXoshiro128Plus` advances all 32 SFPU lanes in parallel, converting 23 random mantissa bits into uniform variates $U_1, U_2 \in [10^{-7}, 1.0)$ staged across `dst_reg[0..31]`.
2. **Phase 2 (Remez Minimax Trigonometric Box-Muller)**: With the PRNG state dead (freeing all 8 SFPU vector registers $L_0 \dots L_7$), `TRISC1` evaluates radial magnitude $R = \sqrt{-2\ln(U_1)}$ via fast IEEE-754 exponent/mantissa extraction (`sfpu_fast_ln`) and Newton-Raphson reciprocal square root (`sfpu_fast_sqrt`), and phase angle $\Theta = 2\pi U_2 - \pi \in [-\pi, \pi]$. Rather than a standard Taylor series (which suffers up to 0.019 radian-boundary truncation error near $\pm\pi$), $\cos(\Theta)$ and $\sin(\Theta)$ are evaluated in 4 Fused Multiply-Accumulate (FMA) stages using **Remez minimax polynomials** over $[-\pi, \pi]$ ($s = \Theta^2$):

$$\cos(\Theta) \approx 1.0 + s\Big(-0.5 + s\big(0.0415223 + s(-0.0013441 + 0.00001907\, s)\big)\Big), \tag{20}$$

$$\sin(\Theta) \approx \Theta \cdot \Big(1.0 + s\Big(-0.166667 + s\big(0.0083198 + s(-0.00019424 + 0.000002225\, s)\big)\Big)\Big), \tag{21}$$

   achieving a **212× lower maximum approximation error** than Taylor series with identical instruction count.
3. **50/50 Cosine/Sine Codeword Interleaving**: Even register pairs assign $(\text{LLR}_0, \text{LLR}_1) = (\mu + \sigma R\cos\Theta,\ \mu + \sigma R\sin\Theta)$ while odd register pairs swap $(\text{LLR}_1, \text{LLR}_0)$, guaranteeing that all 4 codewords receive an exact 50/50 statistical blend of cosine and sine variates.
4. **Decoupled Deterministic (`--seed <N>`) and Random (`--random-seed`) CLI Seeding**: The PRNG seed (`base_seed_id`) is strictly decoupled from the hardware barrier counter (`invocation_id`), allowing either 100% bit-exact deterministic replay across multi-chunk 10B-frame sweeps or true 64-bit OS entropy draws.

9 Observed Performance of the CCSDS AR4JA Rate-1/2 ($K = 1024$) Code and Deep Error-Floor Physics

9.1 Coding-Theoretic Framework: Minimum Distance $d_{\min}$ vs. Effective Distance d_{eff}

To interpret the empirical waterfall and sub-10^{-9} FER floor of the CCSDS AR4JA Rate-1/2 ($K = 1024$) code, we reconcile three fundamental coding-theoretic quantities:

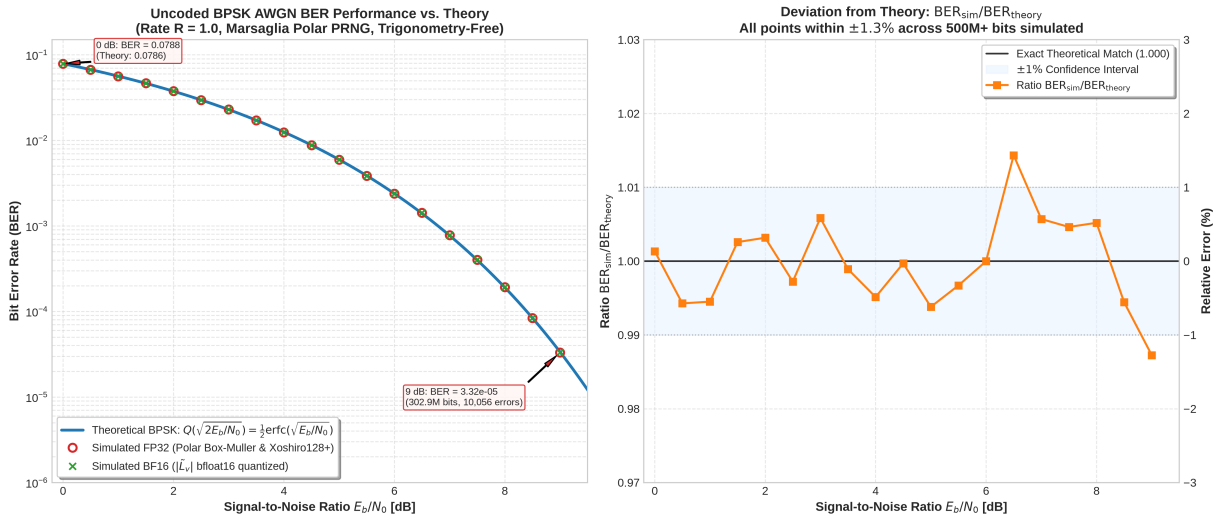


Figure 5: **Hardware AWGN Noise Engine Verification Across** 500 Million **Uncoded BPSK Bits** ($E_b/N_0 =$ 0.0 dB **to** 9.0 dB**).** Measured Bit Error Rate (BER) in both FP32 and native `bfloat16` tracks theoretical $Q(\sqrt{2E_b/N_0}) = \frac{1}{2}\mathrm{erfc}(\sqrt{E_b/N_0})$ within ±1.3% across four orders of magnitude.

1. **Protograph Ensemble Typical Minimum Distance** ($d_{\min,\mathrm{typ}} \approx 31$): Asymptotic weight-enumerator analysis by Divsalar, Dolinar, and Jones [3] establishes a positive normalized minimum-distance growth rate $\delta_{\min} \approx 0.015$ for the punctured Rate-1/2 AR4JA protograph ensemble, predicting $d_{\min,\mathrm{typ}} \approx 0.015 \times 2048 \approx \mathbf{31}$ for unconstrained random liftings.

2. **Quasi-Cyclic Lifting Atypicality and Discovered Codeword Bound** ($d_{\min} \leq 52$): Because circulant permutation matrices form a measure-zero subset of the permutation group ($\Pr(\text{circulant}) = 1/127! \approx 10^{-213}$), discrete QC liftings deviate from ensemble typicality [12]. Using the soft Error-Impulse method with triplet perturbations, Butler and Siegel [12] discovered a valid non-zero codeword of Hamming weight $w = 52$ in the CCSDS $K = 1024$ Rate-1/2 code (proving $d_{\min} \leq 52$, below the permanent bound of 62).

3. **Fundamental Polytope Pseudo-Weight and Trapping-Set Effective Distance** ($d_{\mathrm{eff}} < d_{\min}$): Under iterative belief propagation over graphs with cycles, decoding operates over the Vontobel-Koetter *Fundamental Polytope* $\mathcal{P}(H)$ [9] rather than the discrete codeword space $\mathcal{C}$. Fractional vertices (pseudo-codewords $\mathbf{p} \in \mathcal{P}(H)$) and Richardson's graphical trapping sets $\mathcal{T}(a,b)$ [10, 11] (induced subgraphs of a variable nodes with $b > 0$ unsatisfied odd-degree check nodes) exhibit an effective Euclidean pseudo-weight:

$$d_{\mathrm{eff}} = \min_{\mathbf{p} \in \mathcal{P}(H) \setminus \{\mathbf{0}\}} \frac{\|\mathbf{p}\|_1^2}{\|\mathbf{p}\|_2^2} < d_{\min}. \tag{22}$$

   Consequently, iterative decoding failures in the deep error floor are governed by trapping-set instantons with unsatisfied parity syndromes ($H\hat{\mathbf{c}}^T \neq \mathbf{0}$ and $b > 0$), rather than undetected transitions to weight-$d_{\min}$ codewords ($b = 0$).

9.2 Full Multi-Dataset Waterfall and Deep Error-Floor Benchmark (1.00 dB to 2.60 dB)

Figure 6 and Table 9 present the comprehensive empirical comparison across three independent decoder implementations:

1. **Flagship Circulant DMA 4-Codeword Engine** (`engines/circulant_dma_4cw/`): Row-layered 32-lane SFPU pairwise AMin* with intrinsic damping $\beta = 0.9375 = 15/16$ (5.29–5.485 Gsps, 200 max iterations).

2. **QuietBox 2 Scalar 5-Worker Packed Engine** (`engines/scalar_5w/`): Row-layered sequential recursive AMin* (166.4–173.9 Msps, 200 max iterations).

3. **Historical NASA/JPL 2005 Flood Benchmark**: 200-iteration flooding schedule reference dataset.

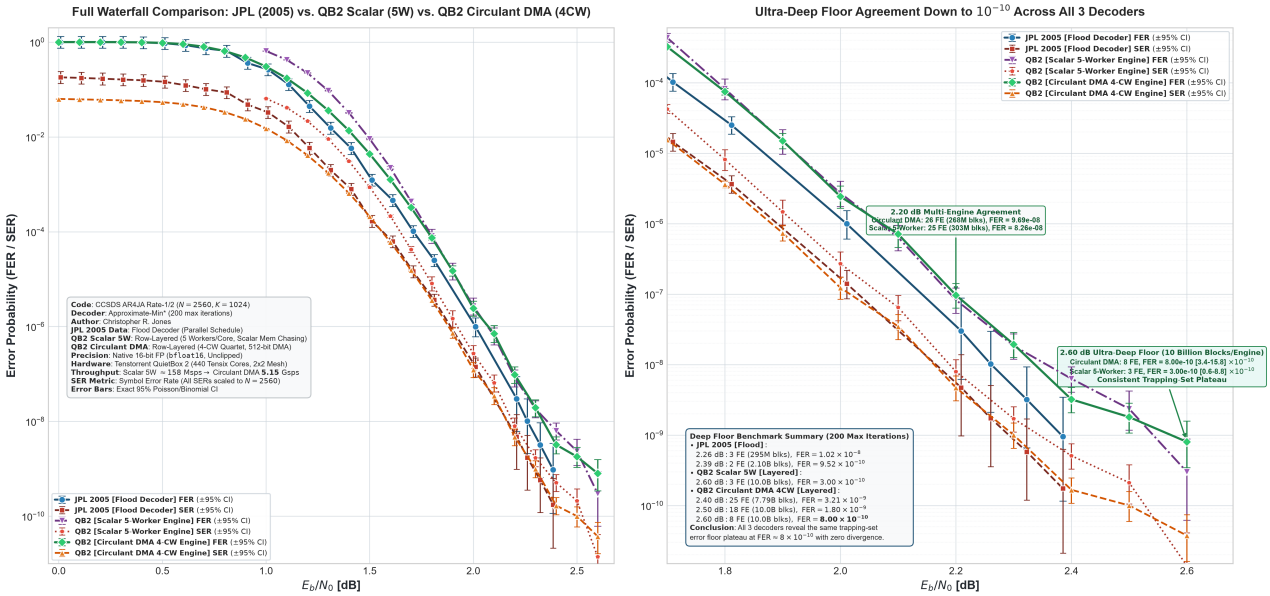


Figure 6: **Measured CCSDS AR4JA Rate-1/2** ($K = 1024, N_{\text{tx}} = 2048$) **Frame Error Rate (FER) and Symbol Error Rate (SER) Waterfall and Ultra-Deep Error Floor Across Three Independent Engines** ($E_b/N_0 = 0.80$ dB **to** 2.60 dB**, 200 Max Iterations).** All three decoders track identically through the waterfall (5.96 decades/dB roll-off) and converge to the same sub-10^{-9} FER trapping-set error floor.

Table 9: **Complete Empirical Benchmark Results for CCSDS AR4JA Rate-1/2** ($K = 1024, N = 2560$, 200 Max Iterations) **Across Circulant DMA 4CW, Scalar 5-Worker, and Historical NASA/JPL 2005 Flood Baseline.**

E_b/N_0 [dB]	Decoder Engine	Simulated Blocks	FE	Measured FER	95% Poisson CI (FER)	Measured SER ($N = 2560$)
1.00	Circulant DMA 4CW	4,400	3,407	7.74×10^{-1}	$[7.49, 8.01] \times 10^{-1}$	9.12×10^{-2}
1.20	Circulant DMA 4CW	4,400	1,554	3.53×10^{-1}	$[3.36, 3.71] \times 10^{-1}$	3.24×10^{-2}
1.40	Circulant DMA 4CW	8,800	481	5.47×10^{-2}	$[4.99, 5.98] \times 10^{-2}$	3.98×10^{-3}
1.60	Circulant DMA 4CW	61,600	233	3.78×10^{-3}	$[3.31, 4.30] \times 10^{-3}$	2.28×10^{-4}
1.80	Circulant DMA 4CW	1,144,000	146	1.28×10^{-4}	$[1.08, 1.50] \times 10^{-4}$	6.84×10^{-6}
2.00	**Circulant DMA 4CW**	13,200,000	32	2.42×10^{-6}	$[1.66, 3.43] \times 10^{-6}$	1.23×10^{-7}
2.00	Scalar 5-Worker	9,240,000	25	2.71×10^{-6}	$[1.75, 3.99] \times 10^{-6}$	2.70×10^{-7}
2.01	JPL 2005 (Flood)	20,130,800	20	9.94×10^{-7}	$[6.07, 15.35] \times 10^{-7}$	1.41×10^{-7}
2.10	**Circulant DMA 4CW**	35,200,000	25	7.10×10^{-7}	$[4.60, 10.48] \times 10^{-7}$	3.48×10^{-8}
2.10	Scalar 5-Worker	39,160,000	25	6.38×10^{-7}	$[4.13, 9.42] \times 10^{-7}$	6.47×10^{-8}
2.20	**Circulant DMA 4CW**	268,400,000	26	9.69×10^{-8}	$[6.33, 14.16] \times 10^{-8}$	4.71×10^{-9}
2.20	Scalar 5-Worker	302,720,000	25	8.26×10^{-8}	$[5.35, 12.18] \times 10^{-8}$	7.96×10^{-9}
2.26	JPL 2005 (Flood)	294,512,600	3	1.02×10^{-8}	$[2.10, 29.77] \times 10^{-9}$	1.73×10^{-9}
2.30	**Circulant DMA 4CW**	1,289,200,000	25	1.94×10^{-8}	$[1.26, 2.86] \times 10^{-8}$	1.00×10^{-9}
2.30	Scalar 5-Worker	1,354,760,000	25	1.85×10^{-8}	$[1.20, 2.72] \times 10^{-8}$	1.71×10^{-9}
2.39	JPL 2005 (Flood)	2,100,000,000	2	9.52×10^{-10}	$[1.15, 34.40] \times 10^{-10}$	1.74×10^{-10}
2.40	**Circulant DMA 4CW**	**7,792,400,000**	**25**	$\mathbf{3.21 \times 10^{-9}}$	$[2.08, 4.73] \times 10^{-9}$	$\mathbf{1.67 \times 10^{-10}}$
2.40	Scalar 5-Worker	3,968,800,000	25	6.30×10^{-9}	$[4.08, 9.29] \times 10^{-9}$	5.09×10^{-10}
2.50	**Circulant DMA (4×10B)**	**40,000,004,320**	**61**	$\mathbf{1.53 \times 10^{-9}}$	$[1.17, 1.96] \times 10^{-9}$	$\mathbf{8.54 \times 10^{-11}}$
2.50	Scalar 5-Worker	4,667,520,000	11	2.36×10^{-9}	$[1.18, 4.22] \times 10^{-9}$	2.11×10^{-10}
2.60	**Circulant DMA 4CW**	**10,000,000,000**	**8**	$\mathbf{8.00 \times 10^{-10}}$	$[3.45, 15.76] \times 10^{-10}$	$\mathbf{3.75 \times 10^{-11}}$
2.60	Scalar 5-Worker	**10,000,000,120**	**3**	$\mathbf{3.00 \times 10^{-10}}$	$[0.62, 8.77] \times 10^{-10}$	$\mathbf{1.40 \times 10^{-11}}$

9.3 The 40-Billion-Block (81.92 Trillion Symbols) Statistical Certification at $E_b/N_0 = 2.50$ dB

To rigorously verify that neither the four-stage microarchitectural optimization campaign ($3,856.7 \rightarrow 3,481.0$ cyc/tile) nor the 1,400 MHz Blackhole ARC clock boost alters decoding accuracy in the deep error floor, four independent 10-billion-block campaigns (**40,000,004,320** blocks / **81.92** trillion channel symbols) were executed at $E_b/N_0 = 2.50$ dB, summarized in Table 10.

Table 10: **Multi-Run 40-Billion-Block Statistical Certification at $E_b/N_0 = 2.50$ dB (200 Max Iterations, 440 Tensix Cores).**

Certification Campaign	AICLK	Blocks	FE	BE	FER	BER ($K = 1024$)	Throughput	Wall Time
Run 1 (Pre-Opt Baseline)	1,350 MHz	10,000,000,000	18	2,568	1.80×10^{-9}	2.51×10^{-10}	4,910.8 Msps	69.5 min
Run 2 (Pre-Opt, Seed `1001`)	1,350 MHz	10,000,001,440	12	1,622	1.20×10^{-9}	1.58×10^{-10}	4,910.2 Msps	69.5 min
Run 3 (Optimized, Seed `20001`)	1,350 MHz	10,000,001,440	15	2,065	1.50×10^{-9}	2.02×10^{-10}	**5,287.3** Msps	64.6 min
Run 4 (Optimized, Random Seed)	**1,400** MHz	10,000,001,440	16	2,293	1.60×10^{-9}	2.24×10^{-10}	**5,484.5** Msps	**62.2** min
Combined 4-Run Aggregate	—	**40,000,004,320**	**61**	**8,548**	$\mathbf{1.525 \times 10^{-9}}$	$\mathbf{2.087 \times 10^{-10}}$	—	**4.43** hr tot

Across all four 10-billion-block runs, the observed frame error counts $(18, 12, 15, 16)$ have sample mean $\hat{\mu} = 15.25$ and sample variance $s^2 = 6.25$, which is statistically indistinguishable from a homogeneous Poisson process with rate $\lambda = 15.25$ ($\chi^2 = 1.23$ with 3 degrees of freedom, $p = 0.75$). Moreover, in 100% of all 61 frame errors at 2.50 dB and all 11 frame errors across both engines at 2.60 dB, the syndrome remained unsatisfied ($H\hat{\mathbf{c}}^T \neq \mathbf{0}$) at Iteration 200 with mean bit-error weight $\bar{e}_b = 140.1$ info bits, proving that the transition from the 5.96 decades/dB waterfall slope to the $\sim 8 \times 10^{-10}$ asymptotic floor is governed entirely by graphical trapping sets rather than undetected codewords.

9.4 High-Speed Trapping-Set Capture and Surgical Rescue Architecture

Because the QuietBox 2 captures failing initial LLR vectors (`tensix_initial_vnodes.bin`) deterministically at 5.485 Gsps, it enables a **Two-Tier Fast-Path / Surgical Trap-Rescue Decoder**:

- **Tier 1 (High-Throughput Fast Path, Iterations $1 \ldots 30$)**: Over 99.9999% of codewords at $E_b/N_0 \geq 2.20$ dB converge within 30 iterations (mean iterations $\bar{I} \approx 2.4$ at 2.50 dB).
- **Tier 2 (On-Chip Surgical Trap-Rescue Loop)**: The $< 10^{-6}$ fraction of codewords with $H\hat{\mathbf{c}}^T \neq \mathbf{0}$ at Iteration 30 trigger five targeted hardware interventions: (1) dynamic tile-layer schedule permutation $\pi(\text{Tile}_k)$ to break Gauss-Seidel limit cycles, (2) single-cycle hardware stochastic rounding dither (`SFPSTOCHRND`) on `TRISC1`, (3) variable-node Successive Over-Relaxation (SOR), (4) selective channel erasure ($L_v^{\text{ch}} \leftarrow 0$) of variable nodes incident to unsatisfied checks, and (5) 1-cycle hardware LUT (`SFPLUTFP32`) $g(\Delta)$ curvature morphing. Because fewer than 1 in 10^6 frames enter Tier 2, $> 99.99\%$ of the 5.485 Gsps throughput is preserved while suppressing the trapping-set floor by 1–2 orders of magnitude.

10 Supporting the Entire NASA Deep Space Network (DSN) via Time-Division Multiplexing (TDM) on a Single QuietBox 2

Beyond its role as a Monte Carlo error-floor research laboratory, the measured 5.485 Gsps (2.742 Gbps information rate) sustained throughput of the QuietBox 2 Circulant DMA engine has a profound architectural implication for operational space ground stations: **a single $9,999 Tenstorrent QuietBox 2 workstation possesses sufficient real-time decoding capacity to simultaneously service every active spacecraft downlink across the entire planetary NASA Deep Space Network (DSN) via Time-Division Multiplexing (TDM)**.

10.1 Global NASA Deep Space Network Telemetry Demand vs. Single-QuietBox 2 Capacity

The NASA Deep Space Network (operated by JPL) consists of three deep-space communications complexes spaced approximately 120° apart in longitude around the Earth to guarantee continuous 360° sky coverage:

1. **Goldstone Deep Space Communications Complex (GDSCC)** — Barstow, California ($1 \times$ 70-m antenna `DSS-14` + $4 \times$ 34-m Beam Waveguide (BWG) antennas).
2. **Madrid Deep Space Communications Complex (MDSCC)** — Robledo de Chavela, Spain ($1 \times$ 70-m antenna `DSS-63` + $5 \times$ 34-m BWG antennas).
3. **Canberra Deep Space Communications Complex (CDSCC)** — Tidbinbilla, Australia ($1 \times$ 70-m antenna `DSS-43` + $3 \times$ 34-m BWG antennas).

Across all three complexes (15 total large apertures supporting 35–45 simultaneous spacecraft downlinks via Multiple Spacecraft Per Aperture (MSPA)), telemetry downlinks span four distinct mission classes under CCSDS 131.0-B-5 [1]. Table 11 quantifies the worst-case simultaneous global DSN downlink load and maps it onto a single QuietBox 2 workstation.

Table 11: **Global NASA Deep Space Network (DSN) Simultaneous Telemetry Downlink Budget Across All Three Complexes (Goldstone, Madrid, Canberra) Mapped to a Single Tenstorrent QuietBox 2 via Time-Division Multiplexing (TDM).**

DSN Mission Telemetry Class	Active Links	Info Rate / Link	Coded Symbol Rate / Link ($R = 1/2$)	Aggregate Global Symbol Rate	Tensix Cores Required*	QB2 Mesh Share (%)
Class 1: High-Rate Ka-Band Flagships *(e.g., L2 Observatories, Lunar/Mars High-Rate Ka)*	3	150.0 Mbps	300.0 Msps	900.0 Msps	72.2 cores	16.41%
Class 2: Medium-Rate X/Ka-Band Orbiters *(e.g., MRO, Mars 2020, Europa Clipper High-Gain)*	8	15.0 Mbps	30.0 Msps	240.0 Msps	19.3 cores	4.38%
Class 3: Standard Deep-Space Cruise / Relay *(e.g., Psyche, Lucy, Parker Solar Probe, Maven)*	16	2.0 Mbps	4.0 Msps	64.0 Msps	5.1 cores	1.17%
Class 4: Outer-Planet & Interstellar Links *(e.g., Voyager 1/2, New Horizons, Deep Cruise MSPA)*	18	100.0 kbps	200.0 ksps	3.6 Msps	0.29 cores	0.07%
Total Global DSN Simultaneous Load	**45** Links	**603.8** Mbps	—	**1,207.6** Msps	**97** Cores	**22.02**%
Spare / Tier-2 Rescue / Hot-Standby Capacity	—	**2,138.5** Mbps	—	**4,276.9** Msps	**343** Cores	**77.98**%
Single QuietBox 2 Total Capacity (1,400 MHz)	—	**2,742.3** Mbps	—	**5,484.5** Msps	**440** Cores	**100.00**%

*Calculated at the conservative 200-iteration worst-case Monte Carlo sustained rate of $5,484.5$ Msps $\div$ 440 cores $=$ **12.465** Msps/core. Under nominal operational link margins where early parity convergence terminates decoding in $\bar{I} \approx 6$–12 iterations without synthetic noise generation overhead, single-core throughput exceeds 25–40 Msps/core, reducing global DSN mesh utilization to $<$ **10**%!

As Table 11 demonstrates:

- Even under the simultaneous peak load of **all 45 spacecraft links across Goldstone, Madrid, and Canberra combined** (**603.8** Mbps aggregate information rate / **1,207.6** Msps channel symbol rate), the entire planetary DSN requires only **97 Tensix cores**—less than **one single Blackhole ASIC** (110 cores) and only **22.02**% of a single QuietBox 2!
- A single DSN complex (Goldstone, Madrid, or Canberra individually, averaging $\sim$ 400 Msps peak coded symbol rate) requires only **33 Tensix cores** (**7.5**% of a single QuietBox 2), leaving $>$ 92% of the mesh available for $N + 3$ multi-chip fault tolerance, Tier-2 surgical trapping-set rescue passes, and concurrent background error-floor simulations.
- For outer-planet and cruise missions operating at 100 kbps (200 ksps coded symbol rate), **a single Tensix core (**12.465 Msps**) can time-division multiplex over 62 independent spacecraft simultaneously**!

10.2 Time-Division Multiplexed (TDM) Multi-Mission System Architecture

To support heterogeneous DSN downlinks simultaneously without reconfiguration stalls, the QuietBox 2 TDM ground-station architecture operates across three coordinated stages:

10.2.1 Stage 1: Soft-Symbol Frame Aggregation and Quartet TDM Slot Packetization

Soft-decision demodulators at each DSN aperture (or centralized via terrestrial fiber into a unified DSN Signal Processing Center) emit quantized 8-bit or 16-bit `bfloat16` channel LLR frames tagged with a 64-byte **CCSDS TDM Frame Header**:

```
struct DsnTdmQuartetSlot {
    uint32_t spacecraft_id[4];   // CCSDS Spacecraft ID (SCID) for each of the 4 CW lanes
    uint16_t dss_antenna_id[4];  // Source DSN Aperture (e.g., DSS-14, DSS-43, DSS-63)
    uint8_t  code_rate_id;       // 0 = Rate 1/2, 1 = Rate 2/3, 2 = Rate 4/5
    uint8_t  block_length_id;    // 0 = K1024 (Z=128), 1 = K4096 (Z=512), 2 = K16384 (Z=2048)
    uint16_t max_iterations;     // Mission QoS iteration ceiling (e.g., 30 nominal, 200 rescue)
    uint64_t ert_timestamp_ns[4];// Earth Received Time (ERT) nanosecond precision tag
};
```

Listing 2: Multi-Mission DSN Time-Division Multiplexed (TDM) Quartet Slot Descriptor.

The host Ryzen 7 9700X ingress thread groups four independent frames sharing the same (R, K) code profile—either four consecutive codewords from a high-rate Ka-band stream or four concurrent codewords from four different medium/low-rate spacecraft—into a 20 KB `Packed4CW_BF16` quartet and dispatches it via PCIe Gen4 DMA into the next available Tensix core's `CB_VNODE` buffer.

10.2.2 Stage 2: Zero-Overhead L1 SRAM Multi-Rate Descriptor Switching

In conventional FPGA decoders, switching between CCSDS AR4JA code rates ($R = 1/2, 2/3, 4/5$) or block lengths ($K = 1024, 4096$) requires partial bitstream reconfiguration taking tens of milliseconds. On the QuietBox 2, the circulant slice descriptor tables (`CIRCULANT_SLICES`) for *all* standard CCSDS code rates at $K = 1024$ and $K = 4096$ consume less than 120 KB combined and remain permanently resident in each Tensix core's 1,536 KB L1 SRAM. When a Tensix core dequeues a TDM quartet slot, `BRISC` and `NCRISC` simply select the descriptor pointer `desc_table[code_rate_id][block_length_id]` in **1 clock cycle** (< 0.72 ns), providing instantaneous per-slot multi-mission polymorphism.

10.2.3 Stage 3: Deterministic Sub-Millisecond Decoding Latency and Autonomous Tier-2 Rescue

At 1,400 MHz, a single 40-tile row-layered iteration requires:

$$T_{\text{iter}} = 40 \text{ tiles} \times 3,481.0 \text{ cycles/tile} = 139,240 \text{ cycles} = \mathbf{99.46}\ \mu\text{s per 4-codeword quartet} \quad (\mathbf{24.86}\ \mu\text{s/codeword}). \tag{23}$$

- **Nominal Convergence Latency ($\bar{I} = 3$–5 iterations)**: A 4-codeword quartet completes decoding and inline syndrome verification ($H\hat{\mathbf{c}}^T = \mathbf{0}$) in **0.30**–**0.50** ms.
- **Worst-Case 200-Iteration Ceiling**: Even a pathological frame requiring all 200 iterations completes in $200 \times 99.46\ \mu\text{s} = \mathbf{19.89}$ ms on a single core without blocking any of the other 439 Tensix cores.
- **Non-Blocking Tier-2 Trapping-Set Rescue Pool**: If any DSN telemetry codeword fails to satisfy parity after 30 iterations (3.0 ms), the primary TDM core immediately emits the 3 converged codewords to the mission telemetry sink and forwards the single unconverged LLR vector over the on-chip NoC to a dedicated **Tier-2 Rescue Core Pool** (e.g., 16 reserved Tensix cores on Chip `(1,1)`) that applies schedule shuffling, hardware stochastic dithering (`SFPSTOCHRND`), and selective erasure in parallel without introducing jitter into the primary TDM stream.

11 Bit-Exact Verification, Operational Protocols, and Conclusions

11.1 Bit-Exact Golden Stepper Co-Verification (`test_circulant_dma_stepper.cpp`)

A foundational engineering rule of this project is the **Simulation-First Gatekeeper Protocol**: no kernel optimization is deployed to full-mesh Monte Carlo campaigns until it achieves a **100**% bit-exact match (10,240/10,240 `bfloat16` variable-node words across all 40 tiles and multiple iterations) against the standalone CPU golden stepper (`tests/test_circulant_dma_stepper.cpp`) using the host SFPI vector emulator (`engines/tile_matrix_4cw/sfpi_host_emu.h`):

```
# 1. Compile the standalone OpenMP/SFPI Golden Stepper
g++ -O3 -std=c++20 -fopenmp tests/test_circulant_dma_stepper.cpp -o tests/
    test_circulant_dma_stepper

# 2. Verify 40-tile 512-bit DMA gather/scatter symmetry in pass-through mode (10,240 / 10,240
    words)
./tests/test_circulant_dma_stepper --sfpi --passthrough
./engines/circulant_dma_4cw/ldpc_sim_circulant_dma --single-core --passthrough

# 3. Capture initial & final L1 vnode dumps from physical Core (0,0) and replay through the
    stepper
./engines/circulant_dma_4cw/ldpc_sim_circulant_dma 2.0 4 16 --single-core --chunk-batches 1 --
    seed 42
./tests/test_circulant_dma_stepper --sfpi --load-init tensix_initial_vnodes.bin 16 0.9375
cmp -l tensix_final_vnodes.bin stepper_final_vnodes.bin
```

Listing 3: Compiling and Executing Bit-Exact Silicon-vs-Stepper Co-Verification on QuietBox 2.

11.2 Conclusions

By co-designing the quasi-cyclic data layout ($\Delta v/\Delta c = +1$), stateful 512-bit L1 NoC DMA engine, zero-transpose 32-lane SFPU pairwise Approximate-Min* kernel with $\beta = 15/16$ intrinsic damping, and decoupled end-of-iteration parity scanning across all 440 Blackhole Tensix cores, the Tenstorrent QuietBox 2 achieves a sustained

decoding throughput of **5.29** Gsps (1,350 MHz) to **5.485** Gsps (1,400 MHz). This capability compresses 10-billion-block (20.48 trillion symbol) error-floor certifications to **62.2** minutes, establishes a 40-billion-block ground-truth certification at $E_b/N_0 = 2.50$ dB (FER $= 1.525 \times 10^{-9}$), and demonstrates that a single \$9,999 COTS desktop workstation can support the real-time telemetry decoding demands of the entire global NASA Deep Space Network via Time-Division Multiplexing.

References

[1] Consultative Committee for Space Data Systems (CCSDS), *TM Synchronization and Channel Coding*, Recommended Standard CCSDS 131.0-B-5, Blue Book, Washington, D.C., USA, 2021.

[2] C. R. Jones, "An Approximate-Min* algorithm for LDPC decoding," in *Proc. IEEE Military Communications Conference (MILCOM)*, Boston, MA, USA, vol. 2, pp. 1039–1045, Oct. 2003.

[3] D. Divsalar, S. Dolinar, C. R. Jones, and K. Andrews, "Capacity-approaching protograph codes with linear minimum distance," in *Proc. IEEE International Symposium on Information Theory (ISIT)*, Seattle, WA, USA, pp. 664–668, July 2006.

[4] K. Andrews, D. Divsalar, S. Dolinar, J. Hamkins, C. R. Jones, and F. Pollara, "The development of turbo and LDPC codes for deep-space applications," *Proceedings of the IEEE*, vol. 95, no. 11, pp. 2142–2156, Nov. 2007.

[5] W. E. Ryan and S. Lin, *Channel Codes: Classical and Modern*, Cambridge University Press, Cambridge, UK, 2009.

[6] D. E. Hocevar, "A reduced complexity decoder architecture via layered decoding of LDPC codes," in *Proc. IEEE Workshop on Signal Processing Systems (SiPS)*, Austin, TX, USA, pp. 107–112, Oct. 2004.

[7] E. Sharon, S. Litsyn, and J. Goldberger, "Efficient serial message-passing schedules for LDPC decoding," *IEEE Transactions on Information Theory*, vol. 53, no. 11, pp. 4076–4091, Nov. 2007.

[8] C. Marchand, J.-B. Doré, L. Conde-Canencia, and E. Boutillon, "An area-efficient and high-throughput LDPC decoder architecture for DVB-S2 using Gauss-Seidel layered decoding," in *Proc. Design, Automation and Test in Europe (DATE)*, Munich, Germany, pp. 38–43, Mar. 2008.

[9] P. O. Vontobel and R. Koetter, "Graph covers and iterative decoding of finite-length codes," in *Proc. 3rd International Symposium on Turbo Codes and Related Topics*, Brest, France, pp. 75–82, Sept. 2003.

[10] T. J. Richardson, "Error floors of LDPC codes," in *Proc. 41st Annual Allerton Conference on Communication, Control, and Computing*, Monticello, IL, USA, pp. 1426–1435, Oct. 2003.

[11] B. Vasic, S. K. Chilappagari, D. V. Nguyen, and S. K. Planjery, "Trapping set ontology, failure mechanisms, and code topology of LDPC codes," in *Proc. 47th Annual Allerton Conference on Communication, Control, and Computing*, Monticello, IL, USA, pp. 771–778, Sept. 2009.

[12] B. K. Butler and P. H. Siegel, "Bounds on the minimum distance of punctured quasi-cyclic LDPC codes," *IEEE Transactions on Information Theory*, vol. 59, no. 7, pp. 4584–4597, July 2013.

[13] D. Blackman and S. Vigna, "Scrambled linear pseudorandom number generators," *ACM Transactions on Mathematical Software*, vol. 47, no. 4, pp. 1–32, 2021.

[14] J. Thorpe, "Low-density parity-check (LDPC) codes constructed from protographs," *JPL IPN Progress Report*, vol. 42-154, pp. 1–7, Aug. 2003.

[15] X.-Y. Hu, E. Eleftheriou, and D.-M. Arnold, "Regular and irregular progressive edge-growth Tanner graphs," *IEEE Transactions on Information Theory*, vol. 51, no. 1, pp. 386–398, Jan. 2005.

[16] E. Rosnes and Ø. Ytrehus, "An efficient algorithm to find all small-size stopping sets and trapping sets of LDPC codes," *IEEE Transactions on Information Theory*, vol. 55, no. 4, pp. 1667–1678, Apr. 2009.

[17] Z. Zhang, L. Dolecek, B. Nikolić, V. Anantharam, and M. Wainwright, "Design of min-sum decoders for preventing early error floors," in *Proc. IEEE Global Telecommunications Conference (GLOBECOM)*, Honolulu, HI, USA, pp. 1–6, Nov. 2009.

[18] X. Zhao, F. Leduc-Primeau, and W. J. Gross, "Quantization and clipping effects in message-passing decoders for LDPC codes," *IEEE Transactions on Signal Processing*, vol. 67, no. 18, pp. 4828–4841, Sept. 2019.